Friday 7:44 PM

Uploaded a file

waht is this about

This PDF is a **job description + technical hiring assignment** for an **AI Engineer role at CalQuity**.

In simple terms

They are hiring an **AI Engineer (0–2 years, remote)** to build AI agents and backend systems for financial-data applications. The role involves AI agents, retrieval/RAG, reasoning over data, APIs, data pipelines, and production systems. CalQuity AI Engineer — Job Desc…

The compensation listed is **₹40,000–₹50,000/month + 0.5–1% equity**, with total CTC of **₹8.8–14 lakh**. CalQuity AI Engineer — Job Desc…

The important part: the assessment

You have to build an **AI chatbot for a fictional logistics company called ParcelPilot**.

The chatbot should answer customer/support questions using:

- 📄 Company policies and SOPs
- 📄 Customer-specific agreements
- 📊 Account/order/ticket data in Excel
- 🔎 Document search/retrieval
- 🧮 Structured-data lookup/calculations
- ⚙️ An action such as creating an escalation or updating a ticket

The system must have **at least 3 tools**: document retrieval, structured-data lookup/calculation, and a state-changing action. CalQuity AI Engineer — Job Desc…

They are specifically testing whether you can build an AI agent

For example, a user might ask:

> "Can Northstar cancel ORD-1001 without a cancellation fee?"

Your agent may need to:

Find order → identify account → read Northstar's agreement → check current cancellation policy → determine answer → explain reasoning.

That's a **multi-step agent workflow**, not just a basic chatbot. CalQuity AI Engineer — Job Desc…

There is also an important security requirement

If you build a customer-facing chatbot, **Customer A must not be able to see Customer B's information**.

They explicitly say access controls should be implemented in the **data/tool layer**, not simply through a prompt telling the LLM not to reveal information. CalQuity AI Engineer — Job Desc…

They also care about hallucination/reliability

The documents intentionally contain conflicts:

- Current policies
- Deprecated policies
- Customer-specific contracts
- Historical support answers that may be wrong

Your agent needs to determine **which source should be trusted**, rather than blindly retrieving the first matching document. CalQuity AI Engineer — Job Desc…

That's probably one of the **most important evaluation points**.

What you have to submit

You need:

1. **Public GitHub repository**
2. **Hosted application** — highly preferred
3. **~5-minute demo video**
4. **Architecture note**
5. **Product note**
6. Brief explanation of **which AI coding tools you used** CalQuity AI Engineer — Job Desc…

They also want you to explain **what you would build next** beyond the minimum requirements.

Difficulty level

I would classify this as **moderately difficult for a beginner, but very achievable with your Python/data background**.

The main challenge isn't making a chat UI. It's correctly implementing:

LLM → tool selection → document retrieval → Excel/database querying → source prioritization → reasoning → confirmation → action

You should **not** hard-code answers to their two example questions because they explicitly say they will test other records and questions. CalQuity AI Engineer — Job Desc…

If you're applying for this role, **this assessment is essentially the main technical project you need to complete for the first round.**

Sources

Sources

ther is 8 pages her i need complete maind pao f al 8 pages and completed information extraction and explantion of all 8 pages lets start with page 1

Absolutely. We'll go **page by page**, and for each page I'll give you:

1. **Complete information extracted from the page**
2. **Simple explanation of every section**
3. **What CalQuity actually expects from you**
4. **Important technical requirements**
5. **What you should remember for the assessment**
6. **Any hidden/important implications in the wording**

Page 1 of 8 — Complete Explanation

1. Page 1 overview

Page 1 contains two main things:

- **The AI Engineer job description**
- **The beginning of the first-round AI Agent Assessment**

The job is for an **AI Engineer: AI Systems** at **CalQuity**. CalQuity AI Engineer — Job Desc…

Part A — The AI Engineer Role

What is CalQuity building?

CalQuity is building **AI infrastructure for financial institutions**.

The purpose is to help financial teams:

- Work with large amounts of financial data
- Make decisions faster
- Make better-informed decisions

So this isn't simply an "AI chatbot developer" position.

The company wants someone who can build **real AI systems that work with data and business processes**. CalQuity AI Engineer — Job Desc…

Part B — What will you actually do?

The job description says you will work on four major areas.

1. Build AI agents and multi-step workflows

You will build AI systems that can:

Understand a request → decide what needs to be done → use tools/data → reason over the results → produce an answer or take an action.

This is different from a simple ChatGPT-style question-and-answer application.

For example:

> User asks a financial question
> ↓
> AI identifies required information
> ↓
> Searches documents
> ↓
> Queries structured data
> ↓
> Performs calculation
> ↓
> Reasons over the results

> ↓
> Gives the answer

This is what they mean by **AI agents and multi-step workflows**. CalQuity AI Engineer — Job Desc…

---

2. Retrieval and reasoning systems

You will develop systems that can retrieve relevant information from **large datasets** and then reason using that information.

This is closely related to **RAG (Retrieval-Augmented Generation)**.

But there is an important distinction:

Retrieval alone is not enough.

The system needs to determine:

- Which information is relevant?
- Which source is trustworthy?
- Which information is current?
- Are there conflicting sources?
- What conclusion should be made?

This becomes particularly important in the assessment because their supplied documents intentionally contain outdated and potentially incorrect information.

---

3. APIs, data pipelines and backend systems

You are also expected to understand the engineering side.

That means the AI isn't supposed to exist as just a notebook or prompt.

You may need components such as:

```
User
 ↓
Chat Interface
 ↓
Backend / API
 ↓
AI Agent
 ↓
Tools
 ├── Document Retrieval
```

```
├── Database / Excel Query
└── Action Tool
```

The job description specifically mentions **APIs, data pipelines, and backend systems**.
CalQuity AI Engineer — Job Desc…

---

4. Production reliability

This is a major point.

They expect you to:

- Debug
- Test
- Deploy
- Monitor
- Make the system reliable

So they aren't only evaluating whether you can make something that works once.

They want you to think:

> "What happens if the AI makes a wrong decision?"

> "What happens if two documents conflict?"

> "What happens if the user asks something outside the system's capabilities?"

> "How do I prevent unauthorized data access?"

This becomes very important later in the assessment. CalQuity AI Engineer — Job Desc…

Part C — Small Team = High Ownership

The company explicitly says it is a **small team**.

Therefore, they expect you to take significant ownership.

The description says you should be able to:

Understand the problem → think about the solution → build it → ensure it works reliably in production. CalQuity AI Engineer — Job Desc…

What does "ownership" mean here?

They don't want someone who only waits for detailed instructions such as:

> "Tell me exactly what code to write."

They want someone who can make reasonable decisions independently.

For example:

> "The requirements don't specify whether I should use PostgreSQL or SQLite. I'll choose SQLite for the assessment because it is simple, local, and sufficient for the supplied dataset."

Then you should be able to explain **why** you made that decision.

Part D — What they're looking for

The company lists several qualities.

Strong engineering fundamentals

You should understand things such as:

- Python
- APIs
- Data structures
- Databases
- Data processing
- Error handling
- Testing
- Backend concepts

The assessment is therefore **not purely an LLM/prompt-engineering exercise**.

Curiosity and product thinking

They want you to ask:

> "What problem is the user actually trying to solve?"

rather than simply:

> "How can I use an LLM here?"

This distinction matters.

A technically impressive AI system that doesn't solve the customer's problem is still a bad product.

Ability to handle ambiguous problems

The company deliberately does **not** give you every architectural decision.

You have flexibility in choosing:

- Architecture
- Tools
- Workflow
- Interface
- Additional functionality

They want to see your judgment.

AI agent / retrieval / reasoning experience

Experience with:

- AI agents
- Retrieval systems
- Reasoning systems
- Multi-step workflows

is listed as a **plus**, not an absolute requirement. CalQuity AI Engineer — Job Desc…

This is important because the role is explicitly **0–2 years experience**.

Most Important Takeaways From Page 1

Area	What CalQuity wants
AI	Build useful AI systems, not just prompts
Agents	Multi-step workflows using tools
Data	Work with large/structured datasets

Area	What CalQuity wants
Retrieval	Find relevant information
Reasoning	Make decisions using retrieved information
Backend	APIs, pipelines and backend systems
Engineering	Debug, test and deploy
Reliability	System should work consistently
Ownership	Solve problems independently
Product thinking	Build something useful to users

The key idea

This job is much broader than "build a chatbot."

They are looking for someone who can build an **end-to-end AI application**.

And that is exactly what the assessment on the following pages is designed to test.

One important distinction for your preparation

Don't interpret the assessment as:

> **"Build a ChatGPT clone."**

A better interpretation is:

> **"Build a reliable AI agent that can retrieve information, query operational data, reason across multiple sources, respect permissions, and safely perform actions."**

That distinction will help you understand **all 8 pages** correctly.

Page 1 is now covered completely.

Next we can do **Page 2**, including the **experience requirement, remote/full-time requirement, compensation/equity, and the beginning of the ParcelPilot assessment**.

what ar eteh structual prouch that we are followingtto understand and understaken to the furthure procedure is there thing like to

Yes. There **is a better structured approach** we can follow for all 8 pages. What we did for Page 1 was useful, but we can make the process more systematic so that by the end you understand **the assignment, the requirements, and the implementation procedure** rather than just having page-by-page notes.

I suggest we use this framework for **every page**:

Our 8-page study framework

1. What the page says — Extraction
We first extract the actual information from the PDF without adding assumptions.

2. What it means — Explanation
Translate the company's wording into simple language.

3. Why it matters — Assessment relevance
Explain why CalQuity included that requirement and what they are likely evaluating.

4. Requirements — What we must build
Separate:

- **Mandatory**
- **Recommended**
- **Optional**

5. Technical concepts
Identify concepts you need to understand, for example:

- AI agents
- RAG/retrieval
- Tool calling
- APIs
- Structured-data querying
- Access control
- Human confirmation
- Multi-step workflows

6. Dependencies — What comes before what

This is especially important. We'll build a dependency chain rather than treating the 8 pages independently.

For example:

```
JOB REQUIREMENTS
       ↓
ASSESSMENT OBJECTIVE
       ↓
UNDERSTAND DATA PACK
       ↓
CHOOSE CHATBOT TYPE
       ↓
DESIGN ARCHITECTURE
       ↓
BUILD RETRIEVAL
       ↓
BUILD DATA TOOL
       ↓
BUILD ACTION TOOL
       ↓
ADD AGENT / ORCHESTRATION
       ↓
ADD ACCESS CONTROL
       ↓
ADD CONFIRMATION
       ↓
TEST MULTI-STEP QUESTIONS
       ↓
UI / DEPLOYMENT
       ↓
DEMO VIDEO
       ↓
ARCHITECTURE + PRODUCT NOTES
       ↓
SUBMISSION
```

7. Decisions we need to make

Whenever the PDF gives us freedom, we'll explicitly record the decision.

For example:

> Customer-facing chatbot **vs.** internal operations chatbot

Then we'll compare the two and choose one based on the assessment requirements.

8. Final action item

At the end of each page, I'll tell you:

> **"After this page, you should understand X, and the next thing we need to determine is Y."**

I also recommend maintaining a running "Assessment Blueprint"

As we go through Pages 1–8, we'll build one master structure:

Category	What we'll capture
🎯 Objective	What problem are we solving?
👤 Users	Who will use the agent?
📚 Data	What information can it use?
🤖 Agent	What should the AI decide?
🔧 Tools	What tools must exist?
🔐 Security	What access restrictions exist?
🧠 Reasoning	How should sources/conflicts be handled?
⚙️ Actions	What can the agent change?
✅ Confirmation	When must it ask permission?
🖥️ Interface	What UI is required?
🚀 Deployment	What needs to be hosted?
🧪 Testing	What questions must it handle?
🎥 Demo	What must the video show?
📝 Documentation	What notes must be submitted?
📊 Product thinking	What additional problem should we solve?

This will prevent a common mistake: **reading the PDF page-by-page but not understanding how all the requirements connect.**

And importantly, I'll keep **three things separate**:

PDF says → directly supported by the assessment
Our interpretation → what the requirement means technically
Our decision → what we eventually choose to implement

That way we don't accidentally treat our assumptions as CalQuity requirements.

For the next pages, **I'll use this structure consistently.**

what are the " wh " words that you choose to use here

Yes. If you mean the **"WH-question framework"** we can use to understand the assignment, I would use these:

WH word	Question we ask	What it helps us understand
What?	What is required?	Requirements and deliverables
Why?	Why is it required?	Business/technical purpose
Who?	Who will use it?	Customer, support staff, admin, etc.
Where?	Where will the information/action come from?	Documents, Excel, database, APIs
When?	When should something happen?	Time-based rules, confirmation, escalation, SLA
Which?	Which source/tool/approach should we use?	Policy vs agreement, retrieval vs database, etc.
How?	How will we implement it?	Architecture, agent, tools, security
Whose?	Whose data/permissions are involved?	Account ownership and access control

But for this particular assessment, I would add 3 more questions:

1. "If?" — What happens if something goes wrong?
Example: What if two documents conflict?

2. "What if?" — What happens in unusual cases?
Example: What if the customer asks for something unsupported?

3. "What next?" — What happens after the answer?
Example: Should the agent escalate, update a ticket, or create a task?

So our master framework becomes:

> **WHAT → WHY → WHO → WHOSE → WHERE → WHEN → WHICH → HOW → WHAT IF → IF → WHAT NEXT**

This is particularly useful for this assignment because CalQuity isn't only asking **"Can you build an AI chatbot?"** They're testing whether you can reason about the complete system: users, data, sources, tools, permissions, decisions, failures, and actions.

For each of the remaining 7 pages, we can go through these questions systematically and mark **what the PDF explicitly says vs. what we infer vs. what we need to decide**.

solve one problem by useing if , what if, and waht next and at last needed unique it should be universitly acceptable

Yes. If you want to use **IF → WHAT IF → WHAT NEXT** as a structured problem-solving method for the ParcelPilot assessment, we can make it academically and professionally sound.

Let's take **one actual problem from the assessment**:

> **Problem:** A customer asks whether they are eligible for a service credit because a pickup was three hours late due to carrier fault. CalQuity AI Engineer — Job Desc…

1. IF — Establish the conditions

We first identify the conditions that must be true before giving an answer.

IF the agent receives the customer's request:

1. Identify the customer's account.
2. Identify the relevant order/shipment.
3. Retrieve the applicable customer agreement.
4. Retrieve the current service-credit policy/SOP.
5. Check the operational data to confirm the delay and its cause.
6. Compare the facts against the applicable rules.

The important point is that the agent **should not answer solely from the user's statement**. It needs evidence from the supplied data and documents.

2. WHAT IF — Handle uncertainty and exceptions

Now we ask:

What if the general policy says one thing but the customer's contract says something different?

The agent should recognize the conflict and apply the appropriate source hierarchy rather than blindly using whichever document was retrieved first.

This matters because the assessment explicitly says that **customer-specific agreements may override general policies**. CalQuity AI Engineer — Job Desc…

What if the policy document is outdated?

The agent should prefer the current source and avoid relying on deprecated material.

What if the operational data doesn't confirm the carrier fault?

The agent should **not confidently claim that the customer qualifies**. It should explain the uncertainty or escalate for human review.

What if the request requires human judgment?

The agent should escalate rather than inventing an answer. The assessment explicitly requires escalation for requests requiring human judgment or unsupported exceptions. CalQuity AI Engineer — Job Desc…

---

3. WHAT NEXT — Decide the appropriate action

After determining the answer:

If the customer qualifies:

The agent explains:

> The shipment qualifies based on the applicable agreement/policy and the verified operational data.

If an action such as creating an escalation or updating a ticket is required, the agent **prepares the action but does not execute it yet**.

It must ask for explicit confirmation before performing a state-changing action. CalQuity AI Engineer — Job Desc…

If the customer does not qualify:

The agent explains the applicable rule and why the conditions are not satisfied.

If the evidence is conflicting or insufficient:

The agent escalates to human support.

The complete reasoning pattern

```
USER REQUEST
     ↓
IF
Are the necessary facts and sources available?
     ↓
WHAT IF
What happens if sources conflict,
data is missing, or the request is exceptional?
     ↓
WHAT NEXT
Answer / request clarification / escalate / prepare action
     ↓
CONFIRMATION
If an action changes system state,
ask the user for explicit confirmation
     ↓
EXECUTE
Only after confirmation
```

Why this is a strong university-level approach

It separates **three different types of reasoning**:

- **IF = Conditions** → What must be established?
- **WHAT IF = Exceptions** → What can go wrong or create uncertainty?
- **WHAT NEXT = Decision/action** → What should the system do after evaluating the evidence?

And for this particular assessment, it aligns well with the requirements around **multi-step reasoning, source reliability, escalation, access control, and confirmation before actions**. CalQuity AI Engineer — Job Desc…

One correction, though: **IF → WHAT IF → WHAT NEXT is a useful reasoning framework, but it is not by itself a complete AI-agent architecture.** We should use it to reason about each business problem, then translate the result into **agent + tools + data + rules + security + action workflow**.

other then this

Yes. If you want **alternatives to IF → WHAT IF → WHAT NEXT**, there are several stronger frameworks. For a university-level AI engineering assessment, I would not rely on ordinary WH-questions alone.

1. 5W1H — Best for understanding the problem

What → Why → Who → Where → When → How

Example:

- **What** is the customer asking?
- **Why** does the customer need an answer?
- **Who** is authorized to access the information?
- **Where** is the required information stored?
- **When** does the policy/agreement apply?
- **How** should the agent resolve the request?

Best for: requirements analysis.

---

2. Problem → Evidence → Rule → Decision → Action

This is **my strongest recommendation for your ParcelPilot project**.

```
Problem
   ↓
Evidence
   ↓
Applicable Rule
   ↓
Decision
   ↓
Action
```

For example:

Problem:
Customer asks whether a late shipment qualifies for a service credit.

Evidence:
Check order/ticket data to verify the delay and carrier fault.

Rule:
Retrieve the applicable current policy and customer agreement.

Decision:
Determine whether the eligibility conditions are satisfied.

Action:
Answer the customer or escalate. If an actual state-changing action is required, obtain confirmation first.

This maps very naturally to the assessment's requirement for multi-step requests, structured-data lookup, document retrieval, reasoning, and actions. CalQuity AI Engineer — Job Desc…

---

3. Input → Process → Output

This is excellent for **system/technical design**.

```
INPUT
User question
Documents
Account data
Order data
Ticket data

      ↓

PROCESS
Retrieve
Validate
Reason
Calculate
Check permissions

      ↓

OUTPUT
Answer
Escalation
Follow-up
Confirmed action
```

Best for: designing the AI agent architecture.

---

4. Detect → Retrieve → Verify → Decide → Act

This is particularly good for an **AI agent**.

```
DETECT
What does the user need?

↓

RETRIEVE
Find relevant documents/data.

↓

VERIFY
Check authority, freshness,
permissions and conflicts.

↓

DECIDE
Determine the correct response.

↓

ACT
Answer, escalate, or perform
a confirmed action.
```

This framework addresses the assessment's concern that policies can change, customer contracts can override general rules, and historical answers may be incorrect.

CalQuity AI Engineer — Job Desc…

---

5. Normal Case → Exception → Escalation

This is useful for **reliability and risk**.

Normal case:
The evidence is complete and sources agree → answer.

Exception:
Sources conflict, data is missing, or the request is outside policy → investigate further.

Escalation:
The system cannot confidently resolve it → send to human support.

The assessment explicitly requires escalation when human judgment or unsupported exceptions are involved. CalQuity AI Engineer — Job Desc…

Which one should we use?

I would **not choose only one**.

For your complete assessment, use this hierarchy:

Level 1 — Understand the requirement

5W1H

Level 2 — Solve the business problem

Problem → Evidence → Rule → Decision → Action

Level 3 — Design the AI system

Input → Process → Output

Level 4 — Make the agent reliable

Detect → Retrieve → Verify → Decide → Act

Level 5 — Handle uncertainty

Normal Case → Exception → Escalation

This gives us a much stronger structure than simply asking WH questions.

For the remaining 7 pages, I recommend we extract the PDF using these five lenses and build one final architecture from them. That way, by Page 8, you won't just understand what the assignment says—you'll have a logical path from **requirements → problem → architecture → implementation → testing → submission**.

Sources

Sources

page 2

Page 2 of 8 — Complete Extraction & Explanation

Page 2 has **two parts**:

1. Remaining **job details**
2. Start of the **ParcelPilot AI Agent Assessment**

I'll separate **what the PDF says** from **what it means for us**.

---

1. Experience, Location & Employment

What the PDF says

- **Experience:** 0–2 years
- **Location:** Remote
- They value **ownership, curiosity, and product judgment** as much as technical skill.
- They are looking for people to join **full time**. CalQuity AI Engineer — Job Desc…

What this means

This is important because you don't need to be a senior AI engineer.

The assessment is likely designed to determine whether someone with relatively limited professional experience can:

- Understand an ambiguous problem
- Make sensible technical decisions
- Build a working AI system
- Explain those decisions
- Think about reliability and users

So the assessment is not simply testing how many AI libraries you know.

---

2. Compensation

The PDF lists:

- **Cash:** ₹40,000–₹50,000 per month
- **Equity:** 0.5%–1.0%
- **Total CTC:** ₹8.8–14 lakh CalQuity AI Engineer — Job Desc…

Important distinction

The compensation information is **job information**, not part of the technical assessment.

So we don't need to incorporate salary/equity into our application architecture.

---

3. Start of the First-Round Assessment

The assessment is called:

> **AI Agent Assessment: ParcelPilot Customer Support**

This is where the important technical work begins. CalQuity AI Engineer — Job Desc…

---

4. What is ParcelPilot?

ParcelPilot is a **B2B logistics platform**.

Businesses use it to:

- Book shipments
- Manage shipments
- Work with multiple carrier partners

It has a **20-person customer operations team** that handles **hundreds of support requests every week**. CalQuity AI Engineer — Job Desc…

Why is this information given?

It establishes the **business context**.

We aren't building a generic chatbot.

We're building an AI support system for a logistics company.

---

5. What kinds of questions do customers ask?

The support team receives questions involving:

- Account entitlements
- Customer-specific contract terms
- Shipment cancellations
- Service credits
- Support SLAs
- Product issues

Some product issues require investigation using:

- Order data
- Account data
- Ticket data CalQuity AI Engineer — Job Desc…

This is extremely important

Notice that the answers don't necessarily exist in one place.

A single question could require:

```
Customer Question
       ↓
Customer Agreement
       +
Current Policy
       +
Order Data
       +
Ticket Data
       ↓
Final Answer
```

This is the foundation for the **multi-step agent** they want us to build.

---

6. Current problem at ParcelPilot

Currently, customer operations employees manually search across:

- Policies
- Product documentation
- Customer agreements
- Previous support tickets
- Structured operational data

to answer customer requests. CalQuity AI Engineer — Job Desc…

What problem are we solving?

The problem is not simply:

> "Customers need a chatbot."

The deeper problem is:

> **Support employees have to manually combine information from multiple sources to resolve customer issues.**

Our AI system should reduce that manual effort while remaining reliable.

---

7. Two types of AI users

This is one of the most important points on Page 2.

ParcelPilot wants an AI support system with **two possible contexts**:

A. Customer-facing support agent

This agent directly interacts with customers.

Its goal:

> Answer customer questions quickly and reliably.

B. Internal support/operations agent

This agent is used by ParcelPilot employees.

Its goal:

- Investigate issues
- Prioritize problems
- Help operations teams
- Act on support activity

The PDF allows us to choose **either one or both**. CalQuity AI Engineer — Job Desc…

---

8. The data is intentionally imperfect

This is probably one of the **most important sentences in the entire assessment**.

The source material is intentionally imperfect.

There may be:

- Outdated documents
- Customer-specific agreements
- General policies
- Historical ticket resolutions containing incorrect guidance

And the system must **not assume every source has equal reliability**.
CalQuity AI Engineer — Job Desc…

Why did they deliberately do this?

Because they want to test whether your AI agent can handle **conflicting or unreliable information**.

A basic RAG system might do this:

```
Question
 ↓
Retrieve document
 ↓
Give answer
```

But CalQuity wants something closer to:

```
Question
 ↓
Retrieve multiple sources
 ↓
Determine source authority
 ↓
Check freshness
 ↓
Check customer-specific applicability
 ↓
Resolve conflicts
 ↓
Reason
 ↓
Answer / Escalate
```

This is a major design requirement.

---

9. What does "customer-specific agreement" mean?

Suppose the general ParcelPilot policy says:

> Cancellation has a fee.

But Northstar Logistics has a special contract saying:

> Northstar may cancel under specific conditions without that fee.

Then the agent cannot simply retrieve the general policy and answer:

> "You must pay the cancellation fee."

It needs to consider the **customer's agreement**.

This becomes particularly important later when the assessment asks:

> "Can Northstar cancel ORD-1001 without a cancellation fee?"

That question is deliberately designed to require reasoning across sources.

---

10. Page 2 — Problem Structure

We can now represent the problem using the framework we discussed.

Problem

Support agents manually search multiple information sources to answer customer questions.

↓

Evidence

The system has:

- Policies
- SOPs
- Product documentation
- Customer agreements
- Account/order/ticket data

↓

Rule

Different sources have different:

- Authority
- Freshness
- Reliability
- Customer applicability

↓

Decision

The AI must determine which information should be used.

↓

Action

It should:

- Answer confidently when possible

- Escalate when human judgment is needed
- Later potentially perform approved actions

This structure is directly supported by the assessment description. CalQuity AI Engineer — Job Desc…

---

11. Page 2 — Key Requirements We Have Discovered

Requirement	Importance
0–2 years	Job eligibility
Remote	Job condition
Full-time	Job condition
B2B logistics context	Business context
Customer support	Main use case
Multiple information sources	Core technical challenge
Customer-specific contracts	Important reasoning requirement
Structured operational data	Data-query requirement
Outdated documents	Reliability challenge
Incorrect historical answers	Hallucination/reliability challenge
Source authority	Must be considered
Customer-facing OR internal agent	Architecture decision
Both contexts allowed	Optional enhancement

---

12. What we should NOT decide yet

At Page 2, **we should not prematurely choose**:

- LLM
- Vector database
- Framework
- Frontend

- Hosting platform
- Database
- Exact agent architecture

The PDF hasn't given us enough information yet.

We should first finish extracting the requirements from **Pages 3–8**.

Only then should we make architecture decisions.

That is important because otherwise we risk choosing technology first and forcing the requirements to fit it.

Page 2 in one sentence

> **ParcelPilot has a support problem where employees manually combine information from policies, contracts, documentation, tickets, and operational data; we need to build an AI agent that can reason across these imperfect sources and reliably help customers or authorized employees.**

Our current understanding

```
PAGE 1
What kind of AI Engineer does CalQuity want?
              ↓
PAGE 2
What real business problem will we solve?
              ↓
PAGE 3
What exact data and constraints are we given?
              ↓
PAGE 4-5
What must the AI system actually do?
              ↓
PAGE 6-7
What additional problems and product thinking are expected?
              ↓
PAGE 8
What exactly must we submit?
```

So **Page 2 establishes the BUSINESS PROBLEM and the DATA/SOURCE COMPLEXITY**. The next page starts giving us the actual **candidate data pack and concrete assessment requirements**.

Sources

Sources

please explain page 2 again

Of course. Let's slow down and explain **Page 2 from the beginning**, in very simple terms. I'll focus on **what the page actually says**, then explain **why each point matters** for your assessment.

Page 2 — Simple, Complete Explanation

Page 2 has two different sections:

1. **Job information**
2. **The beginning of the ParcelPilot assessment**

The second part is much more important for your project.

---

1. Experience: 0–2 years

The job is intended for someone with **0–2 years of experience**. CalQuity AI Engineer — Job Desc…

What does this mean?

CalQuity is not saying:

> "You must already be an expert AI engineer."

Instead, they are looking for someone who can demonstrate good **engineering thinking, curiosity, ownership, and product judgment**.

---

2. Location: Remote

The position is **remote**. CalQuity AI Engineer — Job Desc…

This is simply a job condition. It doesn't affect the technical assessment.

---

3. They care about ownership and curiosity

The company says they care as much about:

- **Ownership**
- **Curiosity**
- **Sound product judgment**

as they do about technical skills. CalQuity AI Engineer — Job Desc…

What does "ownership" mean?

It means they want you to be able to say:

> "Here is the problem. I understand it. Here is my solution. Here is why I chose this approach. I built it and tested it."

They don't want you to simply follow instructions without thinking.

What does "product judgment" mean?

It means thinking about:

> **What should the product actually do for the user?**

Not just:

> "What AI technology can I use?"

---

4. Full-time position

They want people to join **full time**. CalQuity AI Engineer — Job Desc…

Again, this is job information, not part of the chatbot implementation.

---

5. Compensation

The PDF gives:

- Cash: **₹40,000–₹50,000 per month**
- Equity: **0.5%–1.0%**
- Total CTC: **₹8.8–14 lakh** CalQuity AI Engineer — Job Desc…

You don't need to do anything with this information for the assessment.

Now the IMPORTANT part

6. First-round assessment

The technical assignment is called:

AI Agent Assessment: ParcelPilot Customer Support. CalQuity AI Engineer — Job Desc…

They give you a fictional company called **ParcelPilot**.

Think of ParcelPilot as the **customer/business scenario for your AI project**.

---

7. What is ParcelPilot?

ParcelPilot is a **B2B logistics platform**.

Businesses use it to:

- Book shipments
- Manage shipments
- Work with different carrier companies

ParcelPilot has a **20-person customer operations team** handling hundreds of support requests every week. CalQuity AI Engineer — Job Desc…

Simple example

Imagine a company called **Northstar Logistics** uses ParcelPilot.

Northstar might ask:

> "Can we cancel this shipment?"

or:

> "Our pickup was three hours late. Do we get a service credit?"

The ParcelPilot support team has to investigate and answer.

That is the problem your AI system is going to help solve.

---

8. What questions do customers ask?

The PDF gives several examples of information customers may ask about:

Account entitlements

Example:

> "What services are included in our account?"

Customer-specific contract terms

Example:

> "What special terms does our contract with ParcelPilot contain?"

Shipment cancellations

Example:

> "Can I cancel this shipment?"

Service credits

Example:

> "Our shipment was delayed. Are we entitled to a credit?"

Support SLAs

Example:

> "How quickly should ParcelPilot respond to this issue?"

Product issues

Example:

> "Why isn't this feature working?"

Some of these product issues require checking actual operational data.

CalQuity AI Engineer — Job Desc…

---

9. Where is the information stored?

This is **very important**.

The information is not necessarily in one place.

The support team currently searches through:

Documents

- Policies
- Product documentation
- Customer agreements
- Past support tickets

Structured data

- Orders
- Accounts
- Tickets

The employees manually search across all these sources to answer customer questions.

CalQuity AI Engineer — Job Desc…

---

10. So what problem are we actually solving?

Here's the easiest way to understand it:

Current situation

```
Customer asks a question
        ↓
Support employee
        ↓
Search policy
        ↓
Search customer contract
        ↓
Check order
        ↓
Check ticket
        ↓
Compare information
        ↓
Figure out answer
        ↓
Respond to customer
```

This takes time and can lead to mistakes.

What they want

They want an AI system that can help perform this process.

```
Customer
   ↓
AI Agent
   ↓
Search documents
   +
Check structured data
   ↓
Reason over information
```

That's the **core idea of the assessment**.

---

11. There are TWO possible users

This is another very important part.

ParcelPilot wants an AI support system that could have **two different contexts**.

Option A — Customer-facing agent

This talks directly to customers.

Its job is:

> **Answer customer questions quickly and reliably.** CalQuity AI Engineer — Job Desc…

For example:

> Customer: "Can I cancel ORD-1001?"

The agent investigates and responds.

Option B — Internal support/operations agent

This is used by ParcelPilot employees.

Its job is broader:

- Investigate customer issues
- Prioritize issues
- Help operations staff
- Act on support activity CalQuity AI Engineer — Job Desc…

For example:

> Support employee: "Show me all high-severity tickets approaching their SLA."

The internal agent could search the operational data and identify them.

Do we need to build both?

No.

The assessment later explicitly says you need to build **at least one chatbot**, and you can choose either customer-facing or internal.

You can support both if you want. CalQuity AI Engineer — Job Desc…

So we **don't need to decide this yet**.

---

12. Now comes one of the biggest points: the data is imperfect

The assessment deliberately tells you:

> The source material is **not perfect**.

There may be:

- Outdated documents
- Customer-specific agreements
- Incorrect historical support answers

And you must **not assume all sources are equally reliable**. CalQuity AI Engineer — Job Desc…

---

13. Why is this so important?

Imagine you retrieve two documents.

Document A

A general ParcelPilot policy says:

> Cancellation has a fee.

Document B

A customer's specific agreement says:

> This customer can cancel under certain conditions without a fee.

The AI can't simply pick whichever document appears first.

It needs to understand:

> **Which rule applies to this particular customer?**

That's why this assessment is more sophisticated than a simple "upload PDFs and ask ChatGPT questions" project.

---

14. Historical support tickets can also be wrong

Suppose an old support ticket says:

> "Customer received a service credit for this situation."

That doesn't automatically mean the current customer is entitled to one.

The assessment specifically says historical ticket resolutions should be treated as **context only** because they may contain incorrect information. CalQuity AI Engineer — Job Desc…

So:

Past answer ≠ authoritative current policy.

That's a critical concept.

---

15. What information will they give us?

The assessment provides a **Candidate Data Pack**.

It includes several documents and an Excel workbook.

The documents listed are:

1. `01_Support_Policy_v3_CURRENT.pdf`
2. `02_Support_Policy_v2_DEPRECATED.pdf`
3. `03_Cancellation_and_Service_Credit_SOP_v4.pdf`
4. `04_Product_Operations_Guide_and_Known_Issues.pdf`
5. `05_Northstar_Logistics_Enterprise_Agreement.pdf`
6. `06_LumenWorks_Service_Agreement.pdf`
7. `ParcelPilot_Assessment_Data.xlsx` CalQuity AI Engineer — Job Desc…

The Excel workbook contains:

- Account data
- Order data
- Ticket data CalQuity AI Engineer — Job Desc…

So the assignment is intentionally giving you **both unstructured documents and structured data**.

16. One more important rule: time

The workbook contains a **README sheet**.

The assessment says you should use the **dataset snapshot time stated in that README as the reference time for time-based questions**. CalQuity AI Engineer — Job Desc…

This matters because if someone asks:

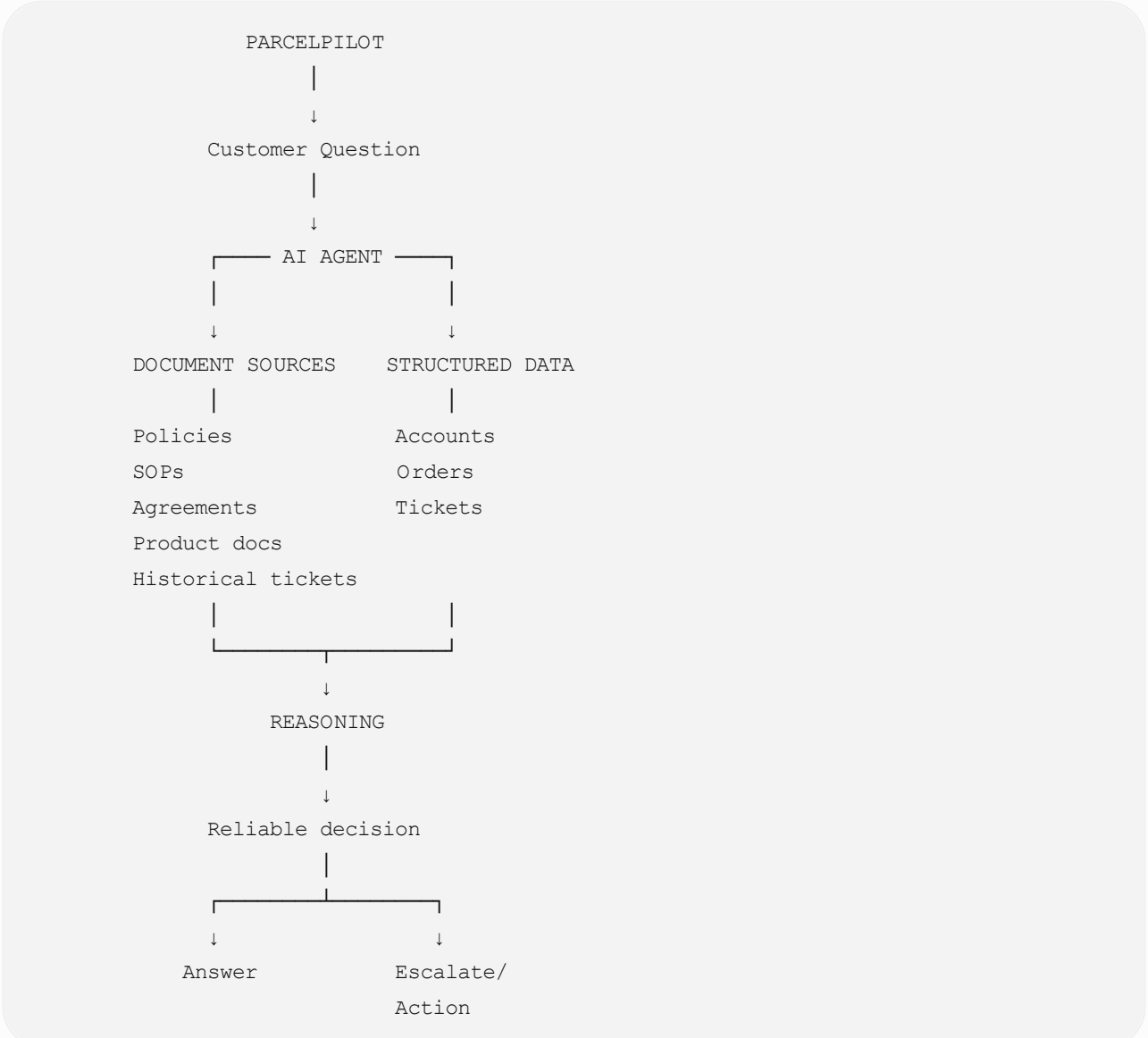
> "Is this ticket currently within SLA?"

you need to know **what date/time the dataset represents**.

You shouldn't simply use today's date.

---

17. The entire Page 2 in one picture



18. The most important concepts from Page 2

If you remember only **7 things**, remember these:

1. ParcelPilot is the business scenario.

2. The problem is customer/support operations.

3. Information exists in multiple sources.

4. Some information is structured data, while some is documents.

5. Sources can conflict or be outdated.

6. Customer-specific agreements can matter differently from general policies.

7. The AI must be reliable rather than simply producing an answer.

The last three points are especially important because **they are likely testing your ability to build a trustworthy agent, not merely a chatbot**. CalQuity AI Engineer — Job Desc…

Our understanding after Page 2

We can now say:

> **The business problem is that ParcelPilot's support team manually combines information from multiple imperfect sources to resolve customer issues. We need an AI agent that can access those sources, determine the relevant information, reason over it, and provide reliable support.**

That's the **core problem**.

Page 3 will give us the exact candidate data pack and the concrete task we are expected to build.

Sources

hwo coaule be knwo that the data is well informative and that agetic ai giving correct answer like rela time employee

Yes. **This is exactly the right question.** A serious AI agent cannot simply assume that its data is correct or that its answer is correct.

For this assessment, we need to distinguish:

> **"The AI found information"** from **"The AI has enough trustworthy evidence to give the correct answer."**

The PDF itself tells us the source data is intentionally imperfect: some documents are outdated, customer agreements may override general policies, and historical ticket resolutions may be wrong. CalQuity AI Engineer — Job Desc…

How can we know the agent is giving a reliable answer?

We should design a **verification pipeline**.

1. First: Know what information is authoritative

Don't treat every document equally.

For example, from Page 2 we already know there are:

- Current policy
- Deprecated policy
- SOP
- Customer agreements
- Product documentation
- Historical tickets

The agent needs a **source hierarchy**.

Conceptually:

```
Customer-specific agreement
          ↓
Applicable current policy / SOP
          ↓
Current product documentation
          ↓
Historical tickets = supporting context only
          ↓
Deprecated documents = do not use as current authority
```

The exact hierarchy should ultimately be based on the supplied data pack and assessment rules, rather than us inventing unsupported rules.

---

2. Verify the customer's identity/context

Suppose the question is:

> "Can I cancel my order?"

The agent shouldn't immediately search for "cancellation policy."

It first needs to know:

```
Who is the customer?
        ↓
Which account?
        ↓
Which order?
        ↓
Which agreement applies?
```

Otherwise, it could apply **Customer A's contract to Customer B**.

This is why the assessment specifically requires access to be scoped so customers cannot see another customer's information. CalQuity AI Engineer — Job Desc…

---

3. Verify the facts using structured data

Suppose the user says:

> "My pickup was three hours late because the carrier caused the delay."

The AI shouldn't simply believe the statement.

It should check:

```
User statement
      ↓
Order data
      ↓
Ticket data
      ↓
Carrier/delay information
      ↓
Verified facts
```

Only then should it apply the relevant policy.

---

4. Compare the evidence against the rule

Now we have:

Facts:

> Shipment was delayed 3 hours and carrier fault is recorded.

Rule:

> What does the applicable current policy/SOP say about service credits?

Customer-specific rule:

> Does this customer's agreement change the general rule?

Then:

```
FACTS + APPLICABLE RULE
          ↓
       DECISION
```

This is much safer than asking an LLM:

> "Does this customer get a credit?"

---

5. What if two sources disagree?

This is where your agent becomes more sophisticated.

Imagine:

Policy: No credit under these circumstances.

Customer agreement: Customer has a special credit entitlement.

The agent should **detect the conflict**, determine which source governs the customer's situation, and explain the basis of its answer.

The assessment explicitly says the system should make deliberate decisions about **source reliability, conflicts, uncertainty, and human intervention**. CalQuity AI Engineer — Job Desc…

---

6. What if the evidence isn't enough?

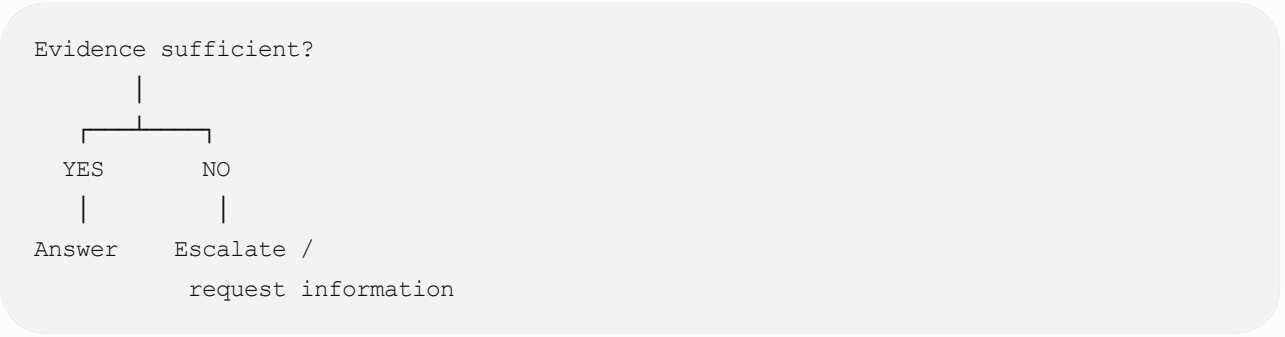
This is critical.

A good employee doesn't say:

> "I don't know, but I'll guess."

A reliable AI agent shouldn't either.

Instead:



The assessment explicitly says queries requiring human judgment, unsupported exceptions, or capabilities outside the system should be escalated. CalQuity AI Engineer — Job Desc…

So **"I cannot determine this reliably" is sometimes the correct AI answer.**

---

7. What about actions?

Suppose the AI concludes:

> "This ticket should be escalated."

It shouldn't automatically create the escalation.

The assessment requires:

Prepare action → ask user for explicit confirmation → execute action.
CalQuity AI Engineer — Job Desc…

For example:

> "I found that the ticket appears to require escalation because X and Y. Would you like me to create the escalation?"

User:

> "Yes."

Only then:

> **Create escalation.**

---

8. How do we know the whole system is actually good?

This is another important distinction.

We need **evaluation**, not just a working demo.

We can create a test set such as:

Test	Expected behavior
Straightforward policy question	Answer
Customer-specific contract	Apply relevant agreement
Deprecated policy	Don't treat as current
Historical incorrect answer	Don't blindly follow it
Conflicting sources	Resolve or escalate
Missing data	Ask/escalate
Unauthorized account	Block access
Action request	Ask confirmation
Multi-step question	Use multiple tools

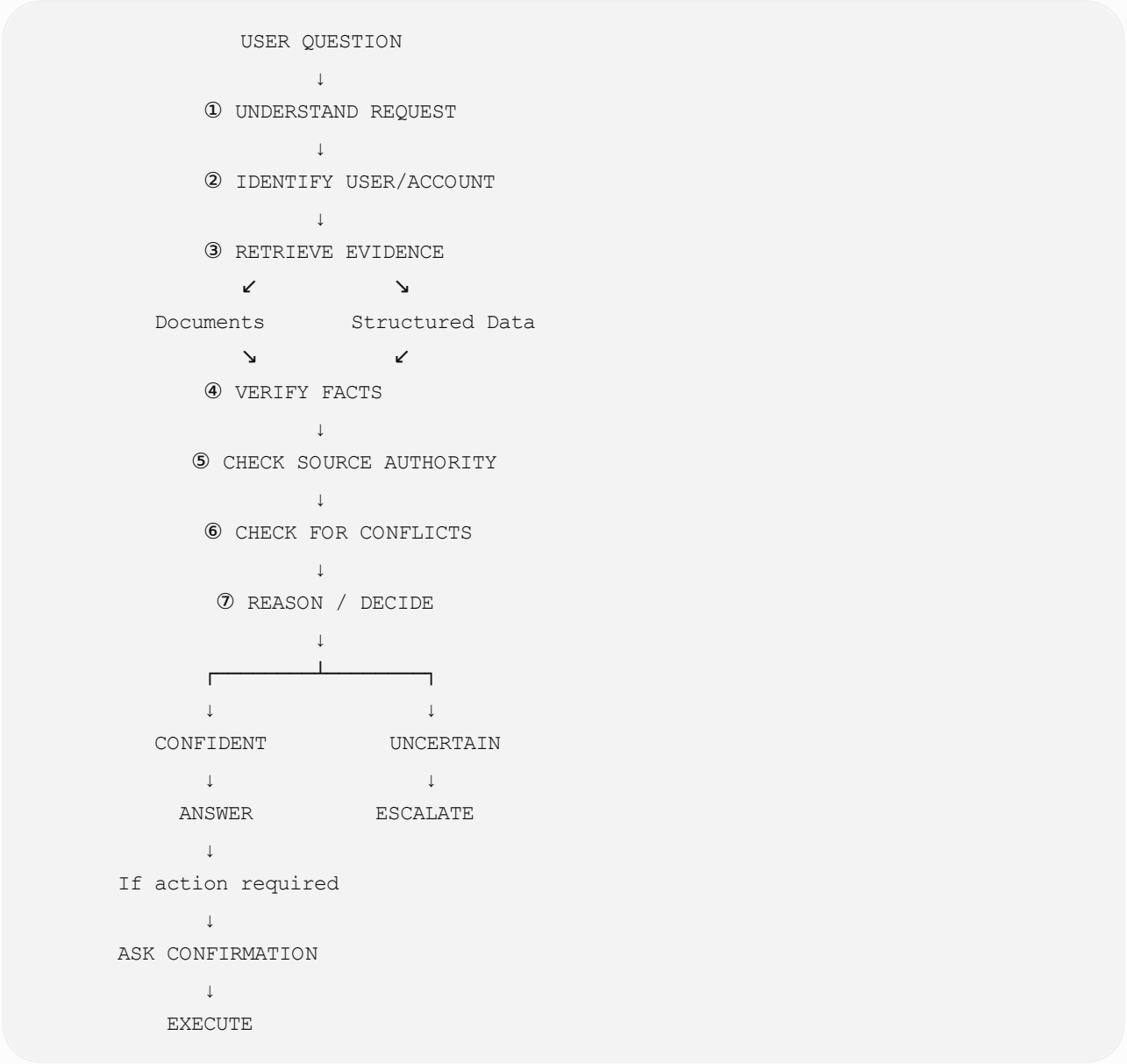
Then we compare:

> **Expected result vs. Agent result**

This is how we begin demonstrating that the agent is reliable rather than merely impressive in a demo.

The complete reliability model

I would use this as our mental model for the project:



The key principle

Don't ask:

> "How do we make the AI sound like a real employee?"

Ask:

> **"How do we make the AI follow a process that a competent employee would use to verify information before making a decision?"**

That is a much stronger engineering approach.

And importantly, **we should not claim that the agent is "correct" merely because the retrieved data looks informative.** We need source authority, verification, conflict handling, escalation, and systematic testing.

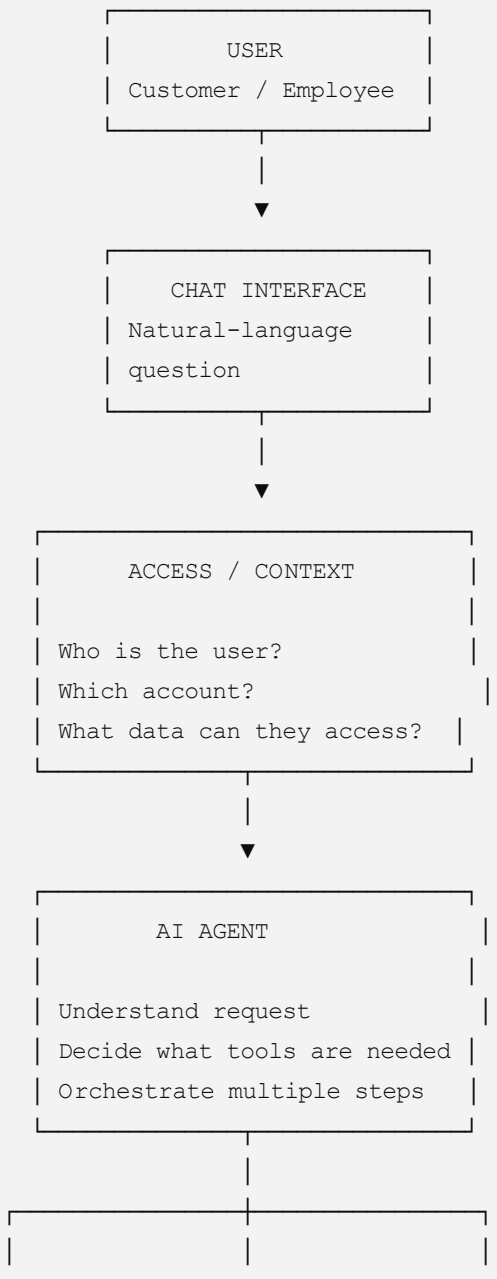
That is exactly the direction the assessment's **Trust and Reliability** requirement is pushing us toward. CalQuity AI Engineer — Job Desc…

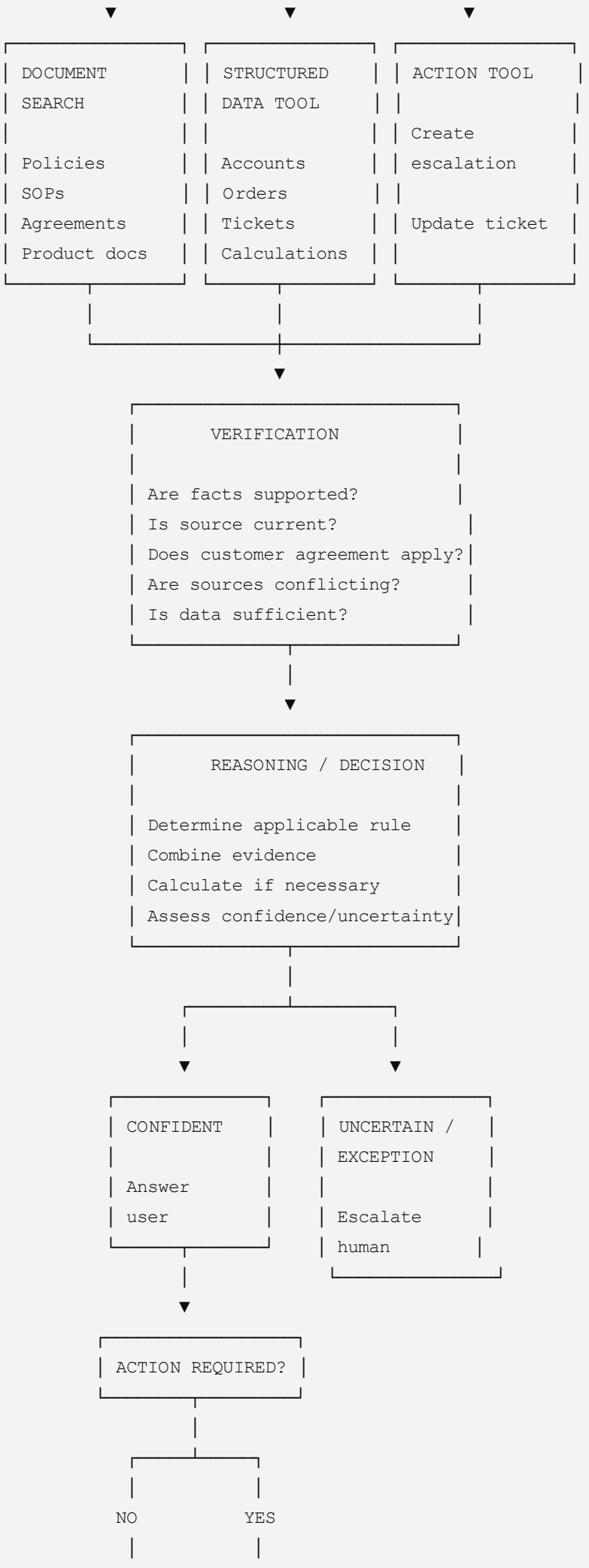
complete map that waht actually the ai system should do

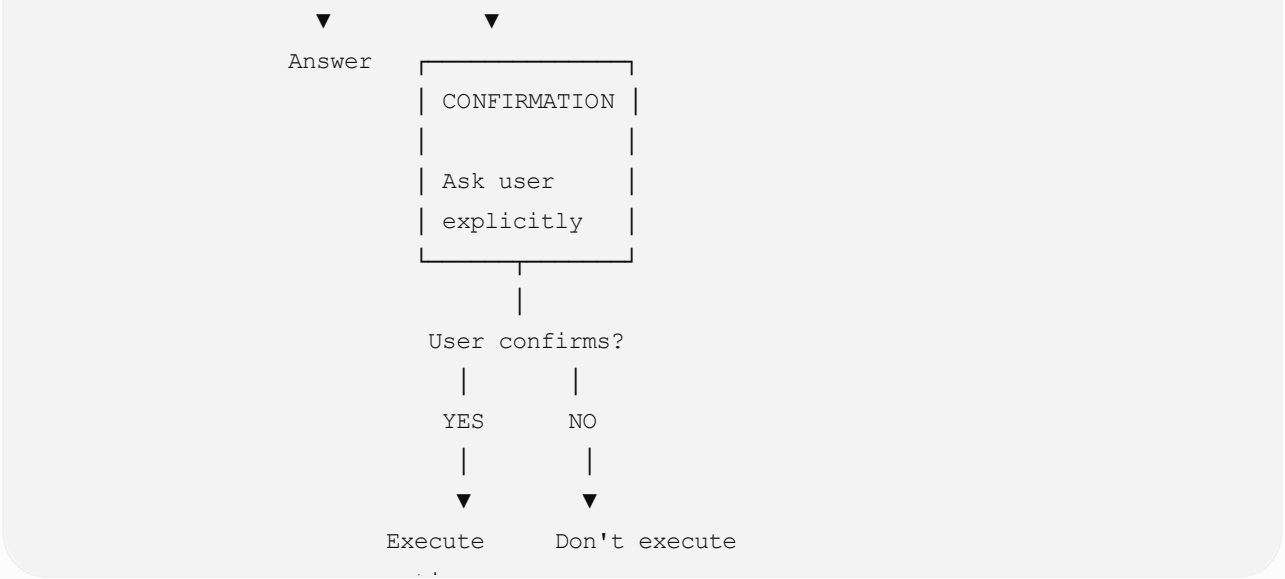
Yes. Based on the assessment, we can now build a **complete map of what the AI system is actually supposed to do**.

The important thing is: **the AI system is not just a chatbot.** It is a controlled decision-support system that can retrieve information, inspect structured data, reason across sources, and—after confirmation—perform actions.

Complete AI System Map




DOCUMENT SEARCH
Policies
SOPs
Agreements
Product docs
STRUCTURED DATA TOOL
Accounts
Orders
Tickets
Calculations
ACTION TOOL
Create escalation
Update ticket
VERIFICATION
Are facts supported?
Is source current?
Does customer agreement apply?
Are sources conflicting?
Is data sufficient?
REASONING / DECISION
Determine applicable rule
Combine evidence
Calculate if necessary
Assess confidence/uncertainty
CONFIDENT
Answer user
UNCERTAIN / EXCEPTION
Escalate human
ACTION REQUIRED?
NO
YES



The assessment explicitly requires a chatbot, access controls, three tool categories, multi-step requests, and confirmation before state-changing actions. CalQuity AI Engineer — Job Desc…

Now let's understand each layer

1. USER

There can be two possible contexts:

Customer

A customer asks something like:

> "Can Northstar cancel ORD-1001 without a cancellation fee?"

Internal employee

A support employee might ask:

> "Which high-severity tickets are approaching their SLA?"

The assessment allows us to build either context, or both. CalQuity AI Engineer — Job Desc…

---

2. CHAT INTERFACE

The user needs a simple chat interface where they can type natural-language requests.

For example:

> "Was ORD-1001 cancelled successfully?"

or:

> "Does this shipment qualify for a service credit?"

The assessment says the interface should ideally show which tool the agent is using.
CalQuity AI Engineer — Job Desc…

---

3. ACCESS CONTROL

Before the AI accesses information, it needs to know:

> **Who is asking?**

and:

> **What are they allowed to see?**

For example:

```
Customer A
   ↓
Can access
Customer A's data

Cannot access
Customer B's data
```

For an internal employee:

```
Employee
   ↓
Check role
   ↓
Determine authorized data
```

The assessment specifically says access control should be enforced in the **data/tool layer**, not merely through instructions to the LLM. CalQuity AI Engineer — Job Desc…

This is a major security requirement.

---

4. AI AGENT

The agent is the **orchestrator**.

It doesn't necessarily answer every question directly.

It determines:

> "What do I need to find out?"

and:

> "Which tool should I use?"

For example:

User asks:

> "Can Northstar cancel ORD-1001 without a fee?"

Agent might reason:

```
I need:
1. Order information
2. Northstar agreement
3. Current cancellation policy/SOP
```

Then it calls the appropriate tools.

That's why the assessment requires **multi-step requests**. CalQuity AI Engineer — Job Desc…

---

5. TOOL 1 — DOCUMENT SEARCH

The first required tool is **document retrieval**.

It searches:

- Policies
- Agreements
- SOPs
- Product documentation
- Other supplied documents CalQuity AI Engineer — Job Desc…

For example:

```
Search:
"Northstar cancellation terms"

        ↓

Northstar Agreement
        +
```

```
Current Cancellation SOP
        +
Relevant Policy
```

---

6. TOOL 2 — STRUCTURED DATA

The second required tool works with the supplied structured data.

It needs to be able to:

- Look up accounts
- Look up orders
- Look up tickets
- Perform calculations

The assessment explicitly requires structured-data lookup or calculation.
CalQuity AI Engineer — Job Desc…

Example:

> "How many hours was this shipment delayed?"

The agent shouldn't guess.

It should query the data.

---

7. TOOL 3 — ACTION

The third required tool performs an action.

Examples given by the assessment include:

- Create escalation
- Update ticket
- Create follow-up task CalQuity AI Engineer — Job Desc…

This is where the system moves from:

> **information retrieval**

to:

> **taking an operational action.**

---

8. VERIFICATION

This is the layer that makes the system **trustworthy**.

The agent needs to ask:

Is the information current?

There is a current policy and a deprecated policy in the supplied data pack. CalQuity AI Engineer — Job Desc…

Is the information applicable?

A customer's individual agreement may matter.

Is the information reliable?

Historical ticket resolutions may contain incorrect guidance. CalQuity AI Engineer — Job Desc…

Are sources conflicting?

If yes, the agent must deliberately handle the conflict rather than blindly choose one.

The assessment specifically identifies **source reliability, conflicts, uncertainty, and human intervention** as trust requirements. CalQuity AI Engineer — Job Desc…

---

9. REASONING / DECISION

Now the AI combines the verified information.

For example:

```
ORDER DATA
+
CUSTOMER AGREEMENT
+
CURRENT POLICY
+
SOP
        ↓
     REASON
        ↓
"Customer qualifies"
```

The important thing is that the final answer should be based on **evidence**, not just the language model's general knowledge.

The supplied data pack should be the information base for the chatbot.
CalQuity AI Engineer — Job Desc…

---

10. ANSWER OR ESCALATE

There are two basic outcomes.

Situation A — Enough evidence

The agent can answer.

> "Yes, based on the applicable agreement and the verified shipment information, the request qualifies."

Situation B — Not enough evidence

The agent should not invent an answer.

It should escalate when:

- Human judgment is required
- An unsupported exception is requested
- The system doesn't have the necessary capability CalQuity AI Engineer — Job Desc…

---

11. ACTION + CONFIRMATION

Suppose the agent determines:

> "This ticket should be escalated."

It **cannot immediately execute the action**.

Instead:

```
Agent prepares escalation
        ↓
"Would you like me to create this escalation?"
        ↓
User confirms
        ↓
Action tool executes
```

This confirmation requirement is explicitly stated in the assessment. CalQuity AI Engineer — Job Desc…

The most important distinction

Think of the system as having **four levels of responsibility**:

Level 1 — FIND

> Where is the information?

Level 2 — VERIFY

> Can I trust and apply this information?

Level 3 — DECIDE

> What conclusion follows from the evidence?

Level 4 — ACT

> Should the system actually change something?

And **Level 4 requires confirmation**.

What the AI should NOT do

This is just as important.

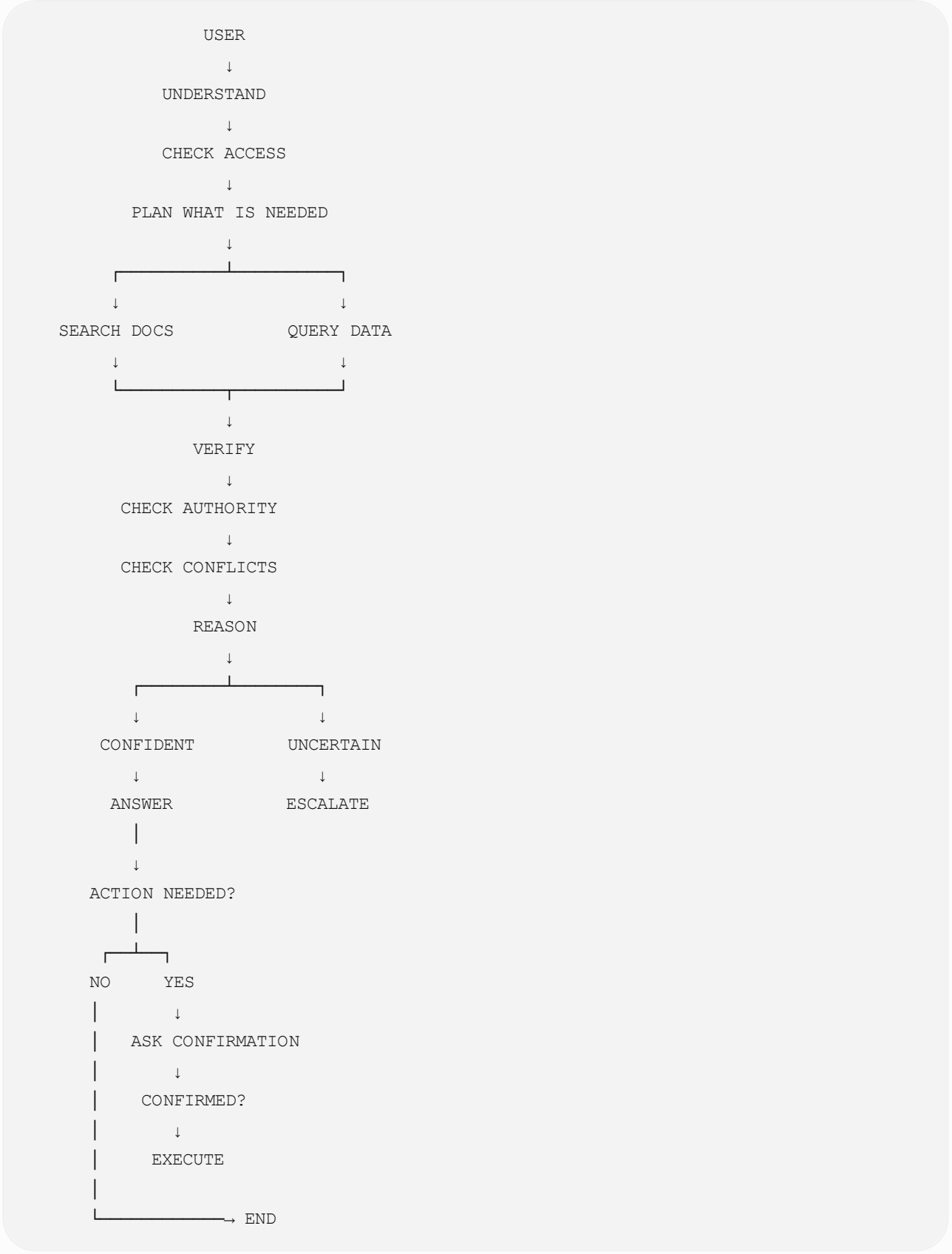
❌ It should NOT:

- Treat every document as equally authoritative.
- Use outdated policies as current rules.
- Treat historical ticket answers as guaranteed truth.
- Allow one customer to access another customer's data.
- Guess when required information is missing.
- Perform state-changing actions without confirmation.
- Hard-code the answers to the two example questions.

The assessment explicitly warns against several of these behaviors. CalQuity AI Engineer — Job Desc…

The final mental model

If you remember only this, remember:



And this is what makes it an AI agent rather than a normal chatbot:

The AI decides what information and tools are needed, performs multiple steps, verifies the evidence, reasons over the results, and chooses whether to answer, escalate, or—after confirmation—take an action.

That is the **core system we should keep in mind while studying Pages 3–8**.

Yes. This point is **very important**, so let's understand it with a simple example.

What does "dataset snapshot time" mean?

The Excel workbook they give you is a **snapshot of ParcelPilot's data at a particular date and time**.

Think of it like taking a photograph of the company's system.

For example, imagine the README says:

> **Dataset snapshot time: January 15, 2026, 12:00 PM**

That means:

> **"For this assessment, pretend the ParcelPilot system/data exists exactly as it looked at January 15, 2026 at 12:00 PM."**

The assessment explicitly tells us to use the snapshot time from the workbook's README for **time-based questions**. CalQuity AI Engineer — Job Desc…

Why is this necessary?

Imagine the data contains:

Ticket	Created	SLA
T001	Jan 15, 10:00 AM	4 hours

Now the user asks:

> **"Is T001 currently within SLA?"**

There are two possible meanings of **"currently."**

Wrong approach ❌

The AI uses the **real-world current date/time**:

> August 22, 2026

That would be wrong because the dataset isn't representing August 2026.

Correct approach ✅

The AI uses the **dataset snapshot time**.

If the README says:

> January 15, 2026, 12:00 PM

then the calculation is:

```
Ticket created:
10:00 AM

Dataset snapshot:
12:00 PM

Elapsed time:
2 hours

SLA:
4 hours

Therefore:
2 < 4
→ Ticket is still within SLA
```

Think of the snapshot as a "frozen clock"

This is the easiest way to remember it.

```
REAL WORLD
August 22, 2026
      ❌
      |
      | Don't use for
      | dataset calculations
      ↓

PARCELPILOT DATA
January 15, 2026
12:00 PM
```

```
      ✅
      |
      ↓
Use this as "NOW"
```

So when the AI answers a question involving **time**, the dataset's snapshot time becomes the system's reference **"now."**

Why would CalQuity do this?

Because otherwise your answer could change depending on **the day you run the application**.

Suppose you submit your project today.

Then the evaluator runs it next week.

If your system uses the computer's actual current time:

```
Today → Ticket within SLA
Next week → Ticket SLA exceeded
```

The exact same dataset could produce different answers.

That's undesirable for an assessment based on a fixed dataset.

Using the snapshot time gives you a **stable reference point**.

Example with a support ticket

Suppose:

Ticket created:
January 15, 2026 — 9:00 AM

SLA:
4 hours

Dataset snapshot:
January 15, 2026 — 11:00 AM

Calculation:

```
11:00 AM
-
 9:00 AM
---------
2 hours elapsed
```

SLA = 4 hours.

Therefore:

> **2 hours < 4 hours → within SLA**

What if the snapshot was 2:00 PM?

Then:

```
2:00 PM
-
9:00 AM
---------
5 hours elapsed
```

SLA = 4 hours.

Therefore:

> **5 hours > 4 hours → SLA exceeded**

Same ticket.

Same dataset.

Different reference time.

That's why the README's snapshot time matters.

How this fits into our AI system

When the user asks:

> "Is this ticket currently within SLA?"

The AI should conceptually do:

```
User Question
      ↓
Identify Ticket
      ↓
Get Ticket Creation Time
      ↓
Get SLA Requirement
      ↓
Get Dataset Snapshot Time
      ↓
Calculate Elapsed Time
      ↓
Compare Elapsed Time vs SLA
      ↓
Answer
```

NOT:

```
Ticket
 ↓
Computer's current date/time
 ↓
Answer
```

One important distinction

The PDF **does not tell us the actual snapshot date/time** in the assessment description.

It tells us:

> **Go to the workbook's README sheet and use the snapshot time stated there.**
> CalQuity AI Engineer — Job Desc…

So we **should not invent a date or time right now**.

When we get to the Excel data pack, we'll inspect the README and record the exact value.

The rule to remember

> **For any time-based question, treat the dataset snapshot timestamp in the README as the reference "now," rather than the actual current date and time.**

This is especially important for questions involving:

- SLA
- Ticket age
- Delays
- Deadlines
- Escalations
- Recent activity
- Time since an order/event

So in our eventual AI system, **the snapshot timestamp should be a system-level reference value used by the data/calculation tool**, not something the LLM is allowed to guess.

Sources

page 3 explanation

Page 3 of 8 — Complete Explanation

Page 3 is where the assessment becomes much more concrete. It tells us:

1. **What data we are given**
2. **What the data contains**
3. **What problems the data has**
4. **What we are actually required to build**
5. **What freedom we have in designing the solution**

Let's go through it carefully.

---

1. Candidate Data Pack

The assessment gives us a **Candidate Data Pack**.

This is the information base that our AI system is expected to work with. The files are:

CalQuity AI Engineer — Job Desc…

Documents

1. `01_Support_Policy_v3_CURRENT.pdf`

This is the **current support policy**.

The filename itself tells us that it is the current version.

---

2. `02_Support_Policy_v2_DEPRECATED.pdf`

This is an **older/deprecated policy**.

This is deliberately important because the agent must not accidentally use an outdated policy as the current rule.

---

3. `03_Cancellation_and_Service_Credit_SOP_v4.pdf`

This is an SOP covering:

- Cancellation
- Service credits

"SOP" means **Standard Operating Procedure**.

This is likely to contain operational rules for handling those situations.

---

4. `04_Product_Operations_Guide_and_Known_Issues.pdf`

This contains product/operations information and known issues.

This can help the agent investigate product-related customer problems.

---

5. `05_Northstar_Logistics_Enterprise_Agreement.pdf`

This is a **customer-specific agreement** for Northstar Logistics.

This is important because a customer's individual agreement may contain terms that differ from general ParcelPilot policies.

---

6. `06_LumenWorks_Service_Agreement.pdf`

This is another customer-specific agreement, this time for LumenWorks.

This gives us another example of customer-specific contractual information.

7. `ParcelPilot_Assessment_Data.xlsx`

This is the structured data workbook. CalQuity AI Engineer — Job Desc…

---

2. What is inside the Excel workbook?

The workbook contains:

- **Account data**
- **Order data**
- **Ticket data** CalQuity AI Engineer — Job Desc…

This is different from the PDF documents.

Think of it like this:

```
DOCUMENT DATA
_____________
Policies
SOPs
Agreements
Product documentation
        ↓
Unstructured / text information


STRUCTURED DATA
_______________
Accounts
Orders
Tickets
        ↓
Rows / columns / records
```

Our AI agent therefore needs to work with **both types of information**.

---

3. Why do we need both documents AND Excel?

Because a customer question may require information from multiple places.

For example:

> **"Can Northstar cancel ORD-1001 without a cancellation fee?"**

The AI might need:

From Excel

> Who owns ORD-1001?

From Northstar agreement

> What special cancellation terms does Northstar have?

From current policy/SOP

> What is the current cancellation rule?

Then it combines those pieces.

```
Order data
    +
Northstar agreement
    +
Current cancellation rules
    ↓
    AI reasoning
    ↓
Final answer
```

The assessment specifically says the system should support requests requiring multiple tools or sources. CalQuity AI Engineer — Job Desc…

---

4. Customer data must be protected

This is a major requirement on Page 3.

The data can be used by:

- Customer-facing agents
- Authorized internal workflows

But customers **must not be able to access information belonging to other accounts**. CalQuity AI Engineer — Job Desc…

Example

Suppose:

```
Northstar
   ↓
ORD-1001
ORD-1002

LumenWorks
   ↓
ORD-2001
ORD-2002
```

If Northstar asks:

> "Show me our orders."

The system can return:

```
ORD-1001
ORD-1002
```

It must **not** return:

```
ORD-2001
ORD-2002
```

This is an **access-control problem**, not merely a prompt problem.

The assessment later explicitly requires access controls to be enforced in the **data/tool layer**. CalQuity AI Engineer — Job Desc…

---

5. The dataset snapshot time

Page 3 repeats an important rule we discussed.

The Excel workbook's **README sheet contains a dataset snapshot time**.

That timestamp must be used as the reference time for **time-based questions**. CalQuity AI Engineer — Job Desc…

So:

```
Real-world current time
        ❌

Dataset snapshot time
        ✅
```

For example, if the user asks:

> "Is this ticket currently within SLA?"

the AI's calculation should use the dataset's defined reference time.

We will need to inspect the workbook itself later to find the **actual timestamp**. The PDF does not provide that timestamp.

---

6. Historical tickets are NOT automatically correct

This is another important rule.

The workbook contains ticket information, and the broader source pack includes historical support resolutions.

But the assessment explicitly says:

> Historical ticket resolutions may contain incorrect information. CalQuity AI Engineer — Job Desc…

Therefore:

Suppose an old ticket says:

> "Customer received a service credit."

The AI should **not automatically conclude**:

> "Therefore every similar customer gets a service credit."

Historical tickets are useful as **context**, not necessarily as authoritative policy.

---

7. What exactly are we being asked to build?

Now we reach the central requirement.

The assessment says:

> **Build at least one AI chatbot for ParcelPilot.** CalQuity AI Engineer — Job Desc…

You have two choices.

Option 1 — Customer-facing chatbot

It answers direct customer questions and escalates requests when necessary.

Example:

> Customer: "Can I cancel ORD-1001?"

The chatbot investigates and answers.

Option 2 — Internal support/operations chatbot

It is used by authorized ParcelPilot employees.

It can:

- Investigate customer issues
- Answer support questions
- Work with operational data

Example:

> Employee: "Find all high-severity tickets approaching their SLA."

Option 3 — Both

The assessment says you are welcome to support **both user contexts**.
CalQuity AI Engineer — Job Desc…

But **both are not mandatory**.

For our eventual implementation, we should decide which option gives the strongest result with reasonable complexity.

---

8. What does "use only the supplied data pack" mean?

This is a very important requirement.

The chatbot should use the **supplied data pack as its information base**.
CalQuity AI Engineer — Job Desc…

That means we shouldn't make the system depend on random internet searches to answer ParcelPilot questions.

For example:

> "What is ParcelPilot's cancellation policy?"

The answer should come from the supplied ParcelPilot documents.

Not:

> Google search → random logistics website → LLM answer.

---

9. But the sources have different authority

The assessment tells us that sources can differ in:

- **Authority**
- **Freshness**
- **Reliability**

The chatbot needs to account for those differences. CalQuity AI Engineer — Job Desc…

This is one of the most important architectural ideas in the entire assignment.

The agent should not think:

> "I retrieved a document, therefore it must be true."

Instead:

> "I retrieved evidence. Now I need to determine whether that evidence is authoritative, current, applicable, and reliable."

---

10. What should happen when the answer is clear?

If the available sources allow the question to be answered **confidently**, the agent should answer directly. CalQuity AI Engineer — Job Desc…

Example:

```
Question
   ↓
Relevant current policy found
   ↓
No conflict
```

```
   ↓
Required data available
   ↓
Answer confidently
```

---

11. What should happen when the question is NOT clear?

If the request requires:

- Human judgment
- An unsupported exception
- Something outside the system's capabilities

the agent should **escalate to the support team**. CalQuity AI Engineer — Job Desc…

This is a critical principle:

> **A good AI agent knows when not to answer.**

We should not optimize the system for "answer every question."

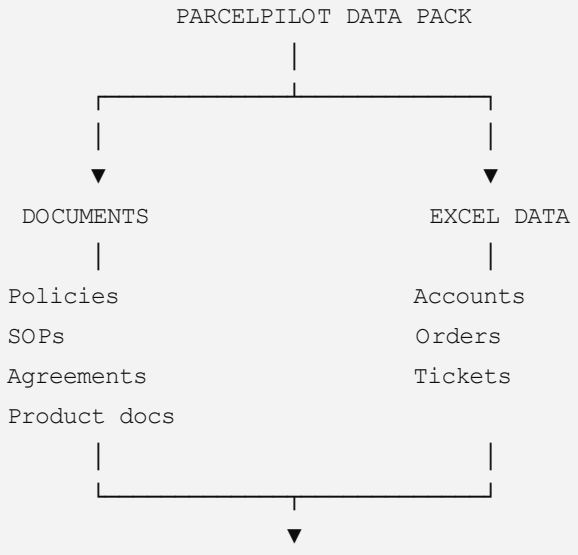
We should optimize it for:

> **Give a reliable answer when evidence is sufficient; otherwise escalate appropriately.**

---

12. What are we actually building?

Putting Page 3 together:



---

13. Page 3 introduces the central design problem

The central challenge is **not simply retrieval**.

It's:

> **How do we make an AI agent reliably reason over imperfect, mixed-format, customer-specific information while respecting access boundaries?**

That's the real engineering problem.

---

14. Page 3 using our structured framework

WHAT?

Build at least one ParcelPilot AI chatbot.

WHO?

Either:

- Customers

- Authorized internal support/operations staff
- Or both

WHAT DATA?

- Policies
- SOPs
- Product documentation
- Customer agreements
- Accounts
- Orders
- Tickets

WHICH SOURCES?

Sources have different authority, freshness, and reliability.

WHAT IF?

- Document is outdated?
- Customer agreement differs from general policy?
- Historical ticket contains incorrect guidance?
- Data is missing?
- User isn't authorized?
- Human judgment is needed?

WHAT NEXT?

- Confident answer → answer
- Insufficient/uncertain/unsupported case → escalate
- Action → later requires confirmation under the assessment's action rules

---

The one thing I want you to understand from Page 3

Page 2 told us the business problem.

Page 3 tells us what information and boundaries our AI system has to work with.

So our understanding is now:

> **We have a chatbot that must reason over both documents and structured data, while respecting customer access, source authority, freshness, and reliability. It should answer when evidence is sufficient and escalate when it isn't.**

That is the foundation we'll use when we move to **Page 4**, where the assessment starts defining the **minimum technical requirements and the three tools the agent must have**.

Uploaded a file

page 4 explanation

Page 4 — Complete Explanation

Page 4 is where the assessment moves from **"What problem are we solving?"** to:

> **"What exactly must our AI agent be capable of doing?"**

This page is especially important because it gives us the **minimum technical capabilities** our system must demonstrate.

---

1. First, understand the difference

So far:

Page 2

We learned the **business problem**.

> Support employees spend time searching different sources to answer customer questions.

Page 3

We learned the **data and constraints**.

> We have PDFs/documents + Excel data, and the information isn't equally reliable.

Page 4

Now we learn:

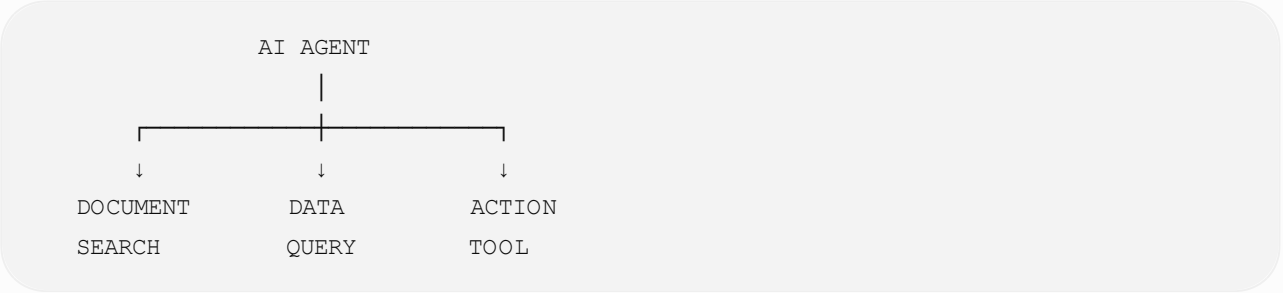
> **What capabilities must the AI agent actually have?**

Think of Page 4 as the **functional specification**.

---

2. The AI must have three main tool categories

The assessment requires the agent to demonstrate three kinds of tools:



These are not optional if you want to satisfy the minimum requirements.

---

3. Tool 1 — Document Retrieval

The first capability is:

> **Search and retrieve information from the supplied documents.**

The agent needs to be able to search things such as:

- Support policies
- SOPs
- Customer agreements
- Product documentation

The assessment specifically requires a document-retrieval tool. CalQuity AI Engineer — Job Desc…

Why do we need this?

Suppose the customer asks:

> **“What is the cancellation policy?”**

The answer isn't supposed to come from the LLM's general knowledge.

The agent should search the ParcelPilot documents.

For example:

```
User question
     ↓
```

```
Agent
     ↓
Document Search
     ↓
Current Cancellation SOP
     ↓
Relevant passage
     ↓
Agent reasoning
```

---

4. But retrieval alone isn't enough

This is a very important distinction.

Imagine the search finds:

v2 — Deprecated Policy

and:

v3 — Current Policy

A basic RAG system might retrieve both and give the LLM both pieces of information.

That's not necessarily safe.

The agent needs to understand:

> **Which source is applicable?**

That's why Page 3's source reliability requirement connects directly to Page 4's retrieval tool.

So:

```
RETRIEVE
   ↓
CHECK SOURCE
   ↓
USE APPROPRIATE INFORMATION
```

Not:

```
RETRIEVE
   ↓
ASSUME TRUE
```

5. Tool 2 — Structured Data Lookup / Calculation

The second required capability is:

> **Access structured data and perform lookups or calculations.** CalQuity AI Engineer — Job Desc…

The workbook contains:

- Accounts
- Orders
- Tickets

So the AI needs a way to query that information.

Example

Customer asks:

> **“What is the status of ORD-1001?”**

The AI shouldn't search the PDF documents for that.

It should use the structured data tool:

```
User
 ↓
"Status of ORD-1001?"
 ↓
Structured Data Tool
 ↓
Order table
 ↓
ORD-1001
 ↓
Status = ...
 ↓
AI response
```

---

6. Calculations are also part of this tool

Suppose the user asks:

> **“Has this ticket exceeded its SLA?”**

The agent may need to calculate:

```
Reference time
-
Ticket creation time
=
Elapsed time
```

Then:

```
Elapsed time > SLA?
```

Therefore:

> Yes → SLA exceeded.

Or:

> No → still within SLA.

And remember our previous discussion:

the dataset snapshot time from the README is the reference time for time-based questions.

So the calculation tool should use that reference rather than blindly using the computer's real-world current time.

---

7. Tool 3 — Action Tool

The third required capability is an **action tool**. CalQuity AI Engineer — Job Desc…

This is different from simply answering a question.

An action means the AI causes something to happen in the support system.

Examples include:

- Create an escalation
- Update a ticket
- Create a follow-up task

Example

Customer asks:

> “Please escalate this issue.”

The AI may determine:

```
User request
     ↓
Check ticket
     ↓
Check severity
     ↓
Check policy
     ↓
Determine escalation is appropriate
```

But it should **not immediately execute the action**.

The assessment requires explicit confirmation before state-changing actions.
CalQuity AI Engineer — Job Desc…

So:

```
AI:
"I've determined that this ticket should be escalated.
Would you like me to create the escalation?"

             ↓

User:
"Yes."

             ↓

Action Tool:
Create escalation
```

---

8. Why are there three tools?

Because real customer-support questions often require **multiple steps**.

Consider:

> **“Can Northstar cancel ORD-1001 without a cancellation fee?”**

The AI may need:

Step 1 — Structured data

Find:

> Who owns ORD-1001?

↓

Step 2 — Document retrieval

Find:

> Northstar's agreement.

↓

Step 3 — Document retrieval

Find:

> Current cancellation rules.

↓

Step 4 — Reason

Compare:

> Order facts + customer agreement + current rules.

↓

Step 5 — Answer

> Yes / No / Uncertain.

That's an **agentic workflow**.

---

9. Multi-step requests

The assessment specifically requires the agent to support **multi-step requests**.
CalQuity AI Engineer — Job Desc…

This means the agent shouldn't be limited to:

> Question → one search → answer.

It should be capable of:

```
Question
   ↓
Understand
   ↓
Tool 1
   ↓
Result
   ↓
Tool 2
   ↓
Result
   ↓
Tool 3 if necessary
   ↓
Reason
   ↓
Answer / Escalate / Action
```

---

10. Example of a complete multi-step question

Let's use:

> **“Can Northstar cancel ORD-1001 without a cancellation fee?”**

The agent's process could be:

Step 1 — Identify the order

Use structured data.

```
ORD-1001
     ↓
Customer = Northstar
```

Step 2 — Retrieve Northstar agreement

Search documents.

```
Northstar agreement
     ↓
Relevant cancellation clause
```

Step 3 — Retrieve current cancellation rules

Search documents.

```
Current SOP/policy
     ↓
Relevant rule
```

Step 4 — Compare

```
Order facts
+
Northstar agreement
+
Current policy
     ↓
Reasoning
```

Step 5 — Answer

If evidence is sufficient:

> Answer the customer.

If the sources conflict or evidence is insufficient:

> Escalate.

This is precisely the kind of workflow the assessment is testing.

---

11. Confirmation before actions

Page 4 also establishes an important safety boundary.

There are two categories:

Read-only

The AI can:

- Search documents
- Read account data
- Read order data
- Read ticket data
- Calculate information

These don't change the system.

State-changing

The AI might:

- Update a ticket
- Create an escalation
- Create a task

These **change system state**.

Therefore:

```
READ
 ↓
Can perform when authorized

WRITE / CHANGE
 ↓
Prepare
 ↓
Ask confirmation
 ↓
Execute only after confirmation
```

This is an important distinction for the architecture.

---

12. Human escalation

The system also needs to know when **not to act**.

If:

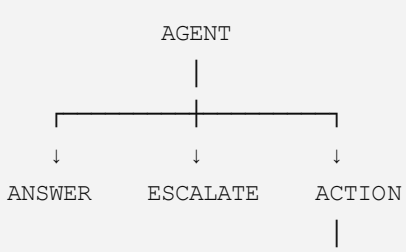
- Evidence is insufficient
- Sources conflict and cannot be resolved
- Human judgment is required
- The request is an unsupported exception

then:

> **Escalate to human support.**

The assessment explicitly requires this behavior. CalQuity AI Engineer — Job Desc…

So the agent has at least three possible outcomes:



```
Confirmation
     |
  Execute
```

---

13. What Page 4 is really testing

The company isn't simply asking:

> “Can you connect an LLM to a PDF?”

They're testing whether you can build an agent that can:

FIND

Find the right information.

QUERY

Retrieve the right structured records.

REASON

Combine multiple pieces of evidence.

VERIFY

Check whether the information is applicable and trustworthy.

DECIDE

Determine what should happen.

ESCALATE

Know when it shouldn't make the decision itself.

ACT

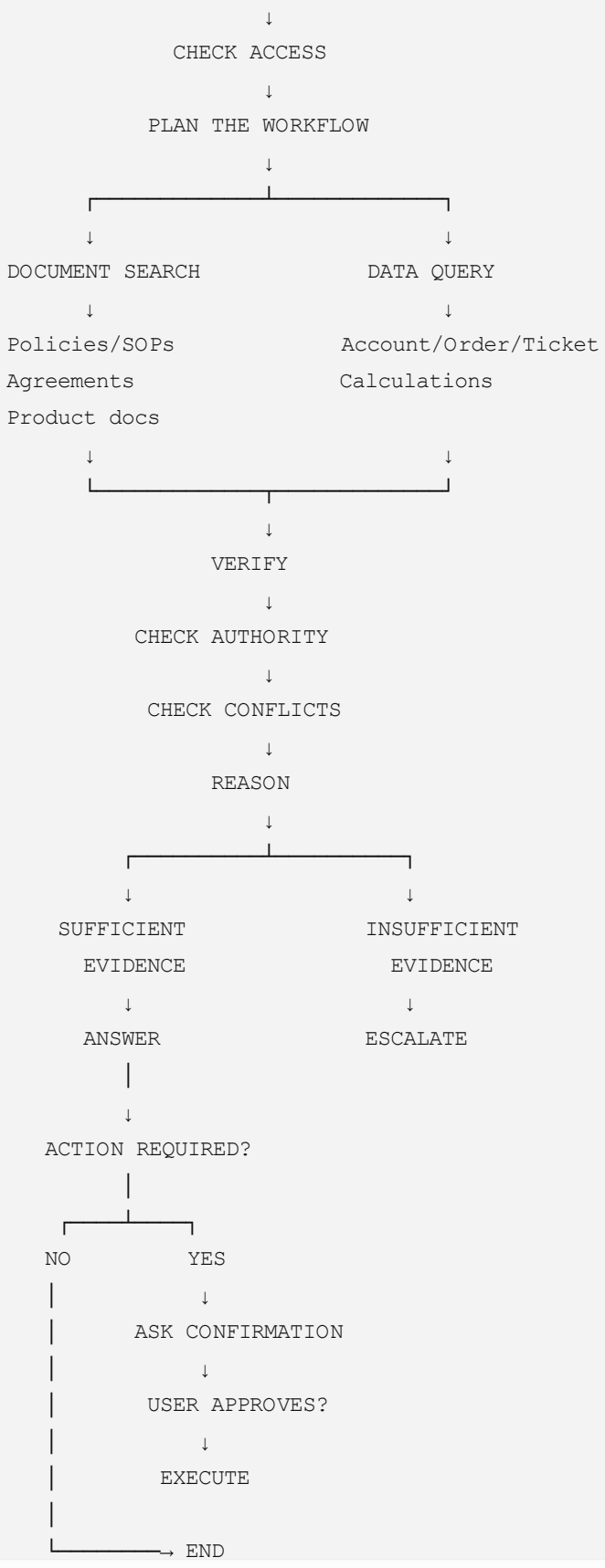
Perform an approved action.

---

14. Page 4 → Complete workflow

Now we can expand our overall map:

```
                USER
                 ↓
        UNDERSTAND REQUEST
```



---

15. The most important thing to remember from Page 4

Three mandatory capabilities:

Capability	Purpose
Document retrieval	Find policies, SOPs, agreements, product information
Structured-data tool	Look up accounts/orders/tickets and calculate
Action tool	Create escalation, update ticket, create task

And the agent must support **multi-step workflows** rather than only one-shot questions.

CalQuity AI Engineer — Job Desc…

One-line understanding of Page 4

> **Page 4 tells us that our AI agent must be able to retrieve documents, query structured data, perform calculations, reason across multiple sources, and take approved actions—with confirmation and escalation mechanisms to prevent unreliable or unauthorized behavior.**

So far:

Page 2 = Business problem
Page 3 = Data + constraints
Page 4 = Required AI capabilities/tools

Page 5 will take us further into the required workflow, reliability, security, and what the final agent needs to demonstrate.

page 5 explanation

Page 5 — Complete Explanation

Page 5 continues from Page 4. **Page 4 told us what tools the agent needs. Page 5 is mainly about how the agent should behave when using those tools—especially reliability, security, multi-step reasoning, and actions.**

I'll keep this strictly tied to the assessment rather than adding requirements that aren't stated.

---

1. The central idea of Page 5

The system should not behave like:

> **Question → LLM → Answer**

Instead, it should behave more like:

```
User question
      ↓
Understand request
      ↓
Check access
      ↓
Retrieve information
      ↓
Check/compare evidence
      ↓
Reason
      ↓
Answer OR escalate
      ↓
If action is needed → confirmation → action
```

This is what makes the system an **agent** rather than a simple question-answering chatbot.

---

2. Multi-step requests

One of the important requirements is that the agent must handle questions that require **multiple steps/tools**. CalQuity AI Engineer — Job Desc…

For example:

> **"Can Northstar cancel ORD-1001 without a cancellation fee?"**

The agent cannot reliably answer from one source.

It may need to:

Step 1 — Find the order

Use structured data.

> Who owns ORD-1001?

Step 2 — Find the agreement

Search the Northstar agreement.

> What cancellation terms apply to Northstar?

Step 3 — Find the current rule

Search the current policy/SOP.

> What is the current cancellation rule?

Step 4 — Compare

```
Order information
+
Northstar agreement
+
Current policy/SOP
```

Step 5 — Decide

Then the agent determines the answer.

That is the kind of **multi-step reasoning** the assessment wants.

---

3. Access control

Another major requirement is **security**.

The agent must not allow one customer to access another customer's information.
CalQuity AI Engineer — Job Desc…

For example:

```
Northstar user
      ↓
Northstar account
      ↓
Northstar orders/tickets
```

They should not be able to ask:

> "Show me LumenWorks' orders."

and receive the information.

Important point

The assessment says access control should be enforced at the **data/tool layer**.

That means we should **not rely only on an instruction to the LLM** such as:

> "Don't show other customers' information."

That's weak security.

The underlying data-access mechanism should actually restrict what the user can retrieve.
CalQuity AI Engineer — Job Desc…

---

4. Why this matters

Imagine an LLM receives:

```
Northstar data
LumenWorks data
Other customer data
```

and we simply tell it:

> "Only show Northstar information."

The model might make a mistake.

A better design is:

```
User
 ↓
Authentication / identity
 ↓
Authorized account
 ↓
Data query restricted to that account
 ↓
Only authorized records returned
 ↓
LLM reasons over permitted information
```

So the model **never needs unrestricted access in the first place**.

---

5. Source reliability

Page 5 also connects strongly to one of the biggest challenges in the assessment:

> **Not all information is equally trustworthy.**

The assessment deliberately gives us:

- A current policy
- A deprecated policy
- Customer-specific agreements
- Historical support information

and tells us that historical resolutions may contain incorrect guidance. CalQuity AI Engineer — Job Desc…

Therefore, our agent needs to consider:

Authority

Is this an official/current rule?

Freshness

Is this document current or deprecated?

Applicability

Does this rule apply to this particular customer?

Reliability

Is this source merely historical context?

---

6. Example of conflicting information

Suppose the agent finds:

Source A

Current general policy:

> Cancellation normally has a fee.

Source B

Northstar agreement:

> Northstar has a special cancellation provision.

Source C

Old support ticket:

> A previous Northstar customer received a fee waiver.

The agent should **not treat all three as equivalent**.

Instead:

```
Current policy
      +
Customer agreement
      +
Historical ticket
      ↓
Evaluate authority/applicability
      ↓
Determine applicable rule
```

The historical ticket should not automatically override the formal sources.

---

7. What if the agent cannot determine the answer?

This is one of the most important safety behaviors.

The agent should not manufacture an answer simply because the user expects one.

If the request requires:

- Human judgment
- An unsupported exception
- Something outside the agent's capabilities

then the appropriate outcome is **escalation**. CalQuity AI Engineer — Job Desc…

So:

```
Evidence sufficient?
        |
    ┌───┴────┐
  YES        NO
```

| |

---

8. Action safety

The agent can have tools that change system state, such as:

- Creating an escalation
- Updating a ticket
- Creating a follow-up task CalQuity AI Engineer — Job Desc…

But there is an important boundary.

The agent must get explicit confirmation before the state-changing action.

For example:

Agent

> "I found that this issue should be escalated. Would you like me to create the escalation?"

User

> "Yes."

Then

> Create escalation.

The sequence is:

```
Analyze
  ↓
Prepare action
  ↓
Ask confirmation
  ↓
User confirms
  ↓
Execute
```

Not:

```
Analyze
  ↓
```

```
Automatically execute
```

The assessment explicitly requires this confirmation behavior. CalQuity AI Engineer — Job Desc…

---

9. Read vs. Write

A useful way to understand this requirement is to divide tools into two categories.

Read operations

These retrieve information:

- Search documents
- Get account
- Get order
- Get ticket
- Calculate

Write operations

These change information:

- Update ticket
- Create escalation
- Create task

The assessment treats state-changing actions more cautiously.

```
READ
 ↓
Retrieve information

WRITE
 ↓
Confirm first
 ↓
Change system
```

---

10. What does "real employee-like" mean here?

This connects directly to the question you asked earlier.

We shouldn't try to make the AI **sound** like an employee.

We should make its **process** resemble a responsible employee's process:

A responsible support employee:

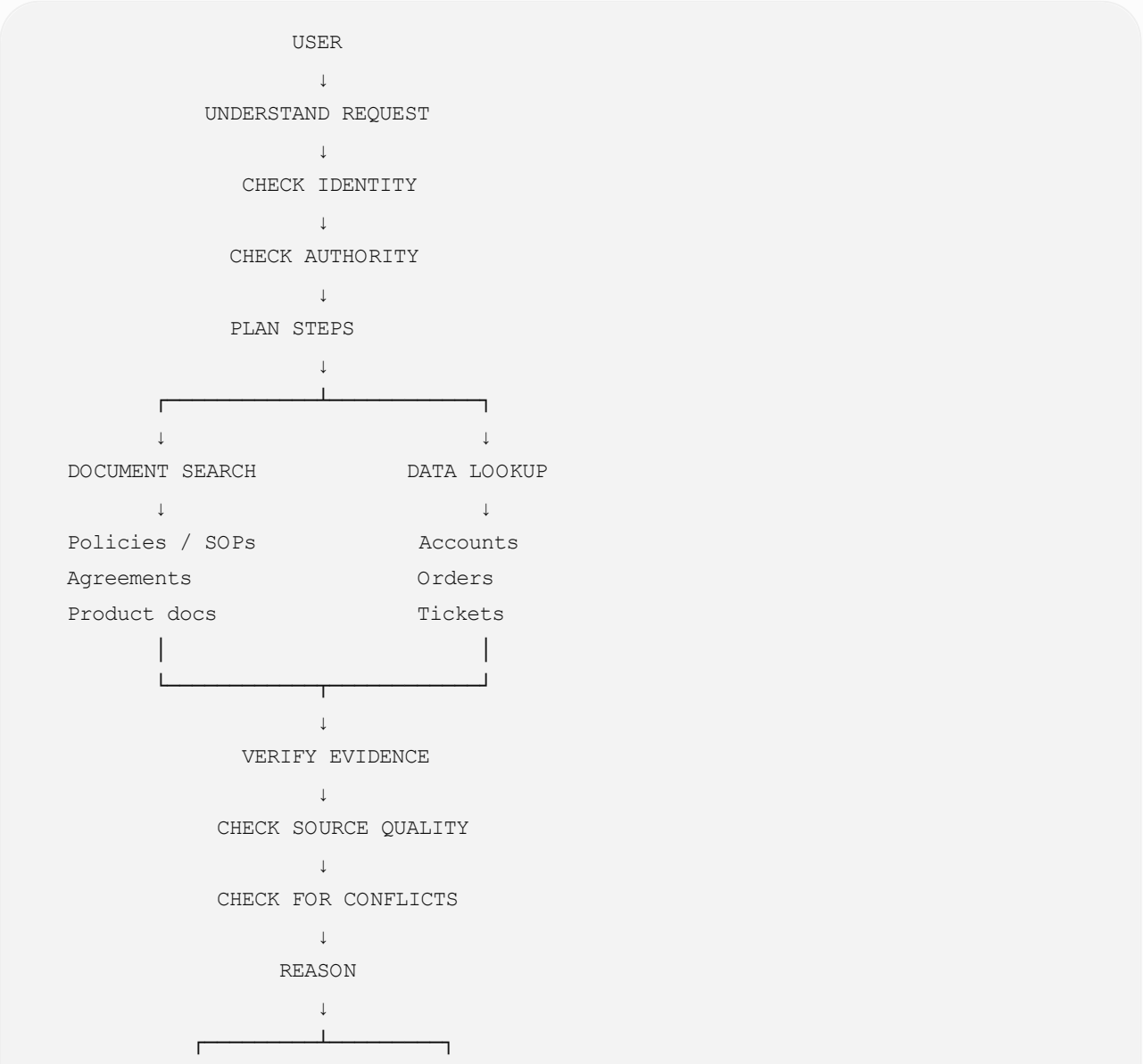
1. Understands the customer's question.
2. Checks the customer's account.
3. Looks at the relevant order/ticket.
4. Checks the applicable policy.
5. Checks customer-specific terms.
6. Doesn't rely blindly on old information.
7. Makes a decision when evidence is sufficient.
8. Escalates when judgment is required.
9. Gets approval before performing a consequential action.

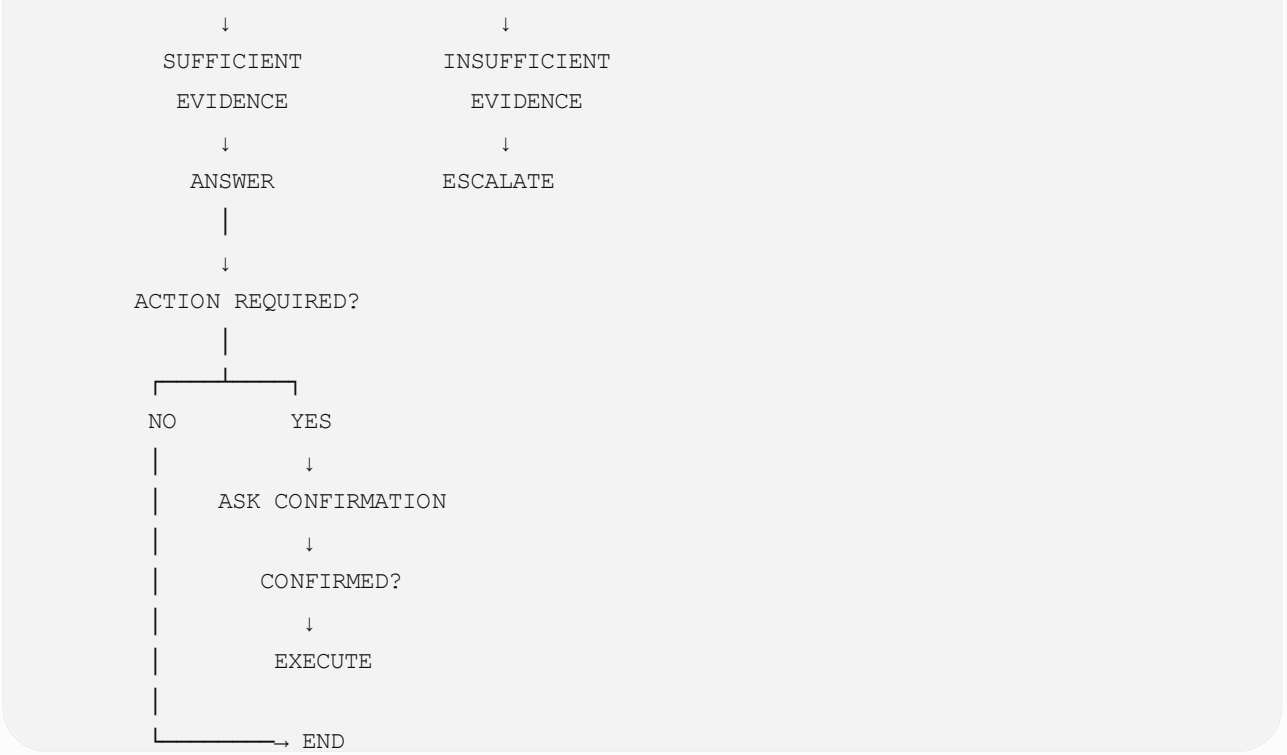
That's much closer to what the assessment is asking us to demonstrate.

---

11. Page 5's complete process

Putting everything together:





---

12. Page 5 using our reasoning framework

Let's apply the framework we decided to use.

WHAT?

What does the agent need to do?

> Retrieve information, query data, reason across multiple sources, answer/escalate, and perform approved actions.

WHO?

Who is asking?

> Customer or authorized internal user.

WHICH?

Which information should be trusted?

> The applicable and sufficiently authoritative/current information.

WHAT IF?

What if:

- Data is missing?
- Sources conflict?

- The information is outdated?
- The request requires human judgment?
- The user isn't authorized?

WHAT NEXT?

> Answer, ask for clarification, escalate, or—after confirmation—execute an action.

---

13. The most important lesson from Page 5

The AI agent is not judged only by whether it produces a correct-looking answer.

It is judged by whether it follows a **safe and defensible process** to reach that answer.

So our eventual system should be able to demonstrate:

> **Who asked → what they need → what evidence was retrieved → why that evidence applies → what decision was reached → whether the system should answer, escalate, or act.**

That is the foundation for making the agent **reliable, explainable, and operationally useful**, rather than just another LLM chatbot.

page 6 explanation

Page 6 — Complete Explanation

Page 6 is mainly about **how the agent should actually behave in real support situations**. At this point, the assessment is moving from the basic architecture toward **trust, reasoning, and practical agent behavior**.

I'll keep this based on the assessment material, and I'll distinguish what is explicitly required from what we are interpreting as the design implication.

---

1. The big idea of Page 6

The system is expected to do more than retrieve an answer.

It should be able to:

> **Understand → Investigate → Verify → Decide → Explain → Escalate or Act**

Think of the AI as a **support employee with controlled tools**, rather than as a search box.

---

2. A customer question may require investigation

Suppose the user asks:

> **"Why hasn't my shipment been completed?"**

The AI may not have the answer in a single document.

It may need to investigate:

```
Customer question
       ↓
Identify account
       ↓
Find order
       ↓
Find related ticket
       ↓
Check product/operations information
       ↓
Reason about the issue
       ↓
Answer
```

This is why the assessment emphasizes **multi-step requests** and multiple tool categories.

---

3. The AI needs to know what information it needs

A strong agent shouldn't blindly call every available tool.

Instead, it should determine:

> **What information is necessary to answer this particular question?**

For example:

Question:

> "What is the cancellation policy?"

Probably requires:

```
Document search
      ↓
Current cancellation policy/SOP
```

But:

Question:

> "Can Northstar cancel ORD-1001 without a fee?"

requires more:

```
Order lookup
      +
Northstar agreement
      +
Current cancellation rule
      ↓
Reasoning
```

So the agent needs to **plan the steps according to the question**.

---

4. The system must respect source quality

This remains one of the most important themes.

The assessment intentionally includes information that may be:

- Current
- Deprecated
- Customer-specific
- Historical
- Potentially incorrect

Therefore, the agent should not simply say:

> "I found a document, so this must be the answer."

Instead, it needs to consider whether the source is actually appropriate.

The assessment explicitly requires deliberate handling of **source reliability, conflicts, uncertainty, and human intervention**. CalQuity AI Engineer — Job Desc…

---

5. Customer-specific information matters

Imagine the general ParcelPilot policy says one thing.

But the customer's agreement contains a special provision.

The agent needs to determine whether that customer-specific agreement affects the answer.

This is why the assessment provides separate agreements for:

- Northstar Logistics
- LumenWorks

These aren't just random PDFs. They are there to test whether the agent can distinguish **general rules from customer-specific terms**.

---

6. Historical tickets should not become "the law"

This is another important trap.

Suppose an old support ticket says:

> "We gave this customer a credit."

The AI should not automatically reason:

> "Therefore the customer is entitled to a credit."

The assessment warns that historical ticket resolutions may contain incorrect guidance. CalQuity AI Engineer — Job Desc…

So historical tickets can help with **context/investigation**, but they shouldn't automatically override formal/current sources.

---

7. The AI needs an uncertainty boundary

This is perhaps the most important behavior for a trustworthy system.

There are cases where the agent simply **cannot determine the answer reliably**.

For example:

```
Required evidence
      ↓
Not available
      ↓
Cannot establish answer
      ↓
Do NOT invent
      ↓
Escalate
```

The assessment explicitly says requests requiring human judgment, unsupported exceptions, or capabilities outside the system should be escalated. CalQuity AI Engineer — Job Desc…

So:

> **"I don't have enough evidence to determine this reliably"**

can be a correct outcome.

---

8. The AI must not cross authorization boundaries

Another major part of the design is access control.

If the user is associated with one customer account, the agent must not expose another customer's:

- Account information
- Orders
- Tickets
- Other private operational information

The assessment requires access control to be enforced at the **data/tool layer**, not merely through an LLM instruction. CalQuity AI Engineer — Job Desc…

So the architecture should look like:

```
USER
 ↓
IDENTITY
 ↓
AUTHORIZED ACCOUNT / ROLE
```

```
 ↓
DATA TOOL
 ↓
ONLY AUTHORIZED RECORDS
 ↓
AI AGENT
```

rather than:

```
USER
 ↓
AI sees everything
 ↓
"Please don't reveal private information"
```

The second approach is weaker.

---

9. Answering versus taking action

Page 6 also connects to an important distinction:

Answering

The AI provides information.

Example:

> "According to the applicable policy, the ticket is outside the SLA."

Acting

The AI changes something.

Example:

> Create an escalation.

These are not equivalent.

For a state-changing action, the assessment requires **explicit user confirmation before execution**. CalQuity AI Engineer — Job Desc…

So:

```
Investigate
   ↓
Determine action
```

```
   ↓
Tell user what will happen
   ↓
Ask confirmation
   ↓
User confirms
   ↓
Execute
```

---

10. Why this makes the agent more like a real employee

Think about how a responsible support employee would work.

They wouldn't normally:

> See a customer's request → immediately change the ticket.

They would:

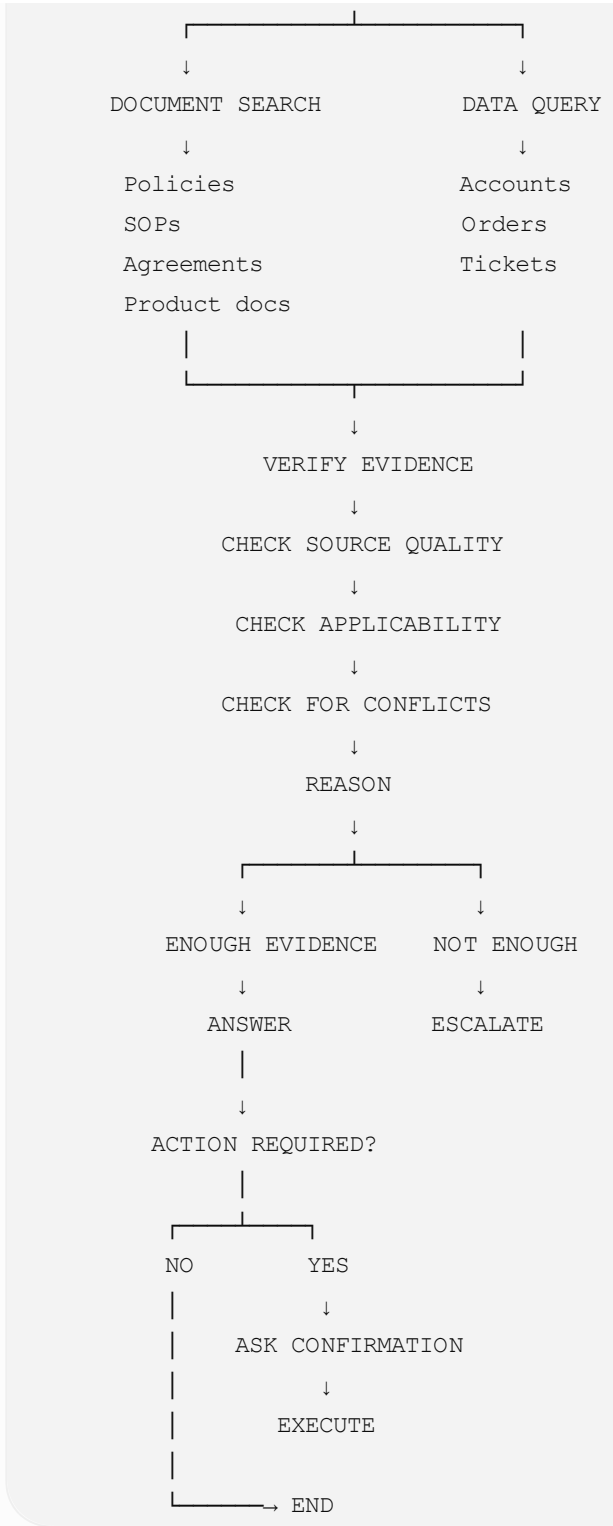
1. Understand the request.
2. Check the account.
3. Investigate the issue.
4. Check the relevant policy.
5. Determine what should happen.
6. Explain the proposed action.
7. Get approval if required.
8. Perform the action.

The assessment is essentially asking us to reproduce this **controlled workflow**, using AI and tools.

---

11. Page 6 — Complete conceptual flow

```
                USER REQUEST
                     ↓
              UNDERSTAND NEED
                     ↓
             IDENTIFY CONTEXT
                     ↓
             CHECK PERMISSIONS
                     ↓
           DETERMINE REQUIRED
                INFORMATION
                     ↓
```



---

12. Page 6 using our framework

WHAT?

What does the user need?

> A support answer, investigation, or operational action.

WHICH?

Which information is relevant?

> The appropriate documents and structured records.

WHO?

Who is the user?

> Customer or authorized internal user.

WHAT IF?

What if:

- Information is missing?
- Sources conflict?
- A source is outdated?
- The request is outside the agent's capabilities?
- Human judgment is required?

WHAT NEXT?

> Answer, ask for clarification, escalate, or prepare an action requiring confirmation.

---

13. The key lesson from Page 6

The assessment is not asking us to build an AI that **always answers**.

It is asking us to build an AI that knows the difference between:

Known

> I have sufficient evidence.

Unknown

> I don't have sufficient evidence.

Restricted

> The user isn't authorized to access this information.

Actionable

> I know what should happen, but I need confirmation before changing system state.

Human judgment required

> This should be escalated.

That distinction is central to building a trustworthy support agent.

In one sentence

> **Page 6 is about making the agent behave responsibly: investigate using the right sources and tools, respect permissions, distinguish reliable evidence from uncertain information, answer when justified, escalate when necessary, and require confirmation before consequential actions.**

So our progression is now:

Page 2 → Business problem
Page 3 → Data + constraints
Page 4 → Required tools/capabilities
Page 5 → Multi-step agent workflow
Page 6 → Reliability, authorization, uncertainty, escalation, and action safety

Sources

Sources

page 7 explantion

Worked for 10s

Page 7 — Complete Explanation

I checked the actual assessment document again. **Page 7 is important because it introduces the two broader client problems that go beyond the minimum chatbot requirements.** CalQuity AI Engineer — Job Desc…

There are **two problems**:

1. **Proactive Issue Detection**
2. **Trust and Reliability**

Let's understand them separately.

1. Problem 1 — Proactive Issue Detection

The normal chatbot is **reactive**.

That means:

> User asks a question → AI responds.

For example:

```
Customer
   ↓
"I have a problem with my shipment."
   ↓
AI investigates
   ↓
AI answers
```

That's useful, but ParcelPilot wants something more.

They want the system to **look at support activity and identify problems before someone specifically asks about them**. CalQuity AI Engineer — Job Desc…

---

2. Reactive vs Proactive

Reactive AI

```
PROBLEM OCCURS
      ↓
Someone notices
      ↓
Someone asks AI
      ↓
AI investigates
```

Proactive AI

```
DATA
 ↓
AI continuously/periodically analyzes patterns
 ↓
Detects unusual or important issue
 ↓
Alerts support team
```

```
  ↓
Team investigates
```

That is a significant change.

The AI is no longer just a **question-answering system**.

It becomes an **issue-detection system**.

---

3. What kinds of problems should it detect?

The assessment gives several examples.

A. Sudden increase in similar complaints

Imagine normally there are:

```
Monday → 3 complaints
Tuesday → 4
Wednesday → 5
Thursday → 25
```

The AI might detect:

> "There has been an unusual increase in complaints related to the same issue."

This could indicate a new product problem.

CalQuity AI Engineer — Job Desc…

B. Multiple tickets about the same product issue

Imagine:

```
Ticket 101 → Product error
Ticket 102 → Product error
Ticket 103 → Product error
Ticket 104 → Product error
```

Instead of treating these as four unrelated tickets, the system could identify:

> **"These tickets appear to be related to the same product issue."**

That gives the operations team a higher-level view.

CalQuity AI Engineer — Job Desc…

C. High-severity tickets approaching or exceeding SLA

This connects directly to what we discussed about the dataset snapshot time.

The system could identify:

```
High severity
      +
SLA approaching
      ↓
URGENT
```

or:

```
High severity
      +
SLA exceeded
      ↓
URGENT ESCALATION
```

The assessment specifically mentions this use case. CalQuity AI Engineer — Job Desc…

D. Unusual patterns

The AI could analyze:

- Orders
- Tickets
- Support activity

and identify something that doesn't look normal.

For example:

> "There is an unusual increase in cancellation requests from a particular customer segment."

The assessment doesn't prescribe exactly how to detect these patterns, so **we should not pretend that a specific algorithm is required**. The requirement is to think about proactive issue detection as an additional product problem. CalQuity AI Engineer — Job Desc…

E. Issues affecting multiple customers

This is particularly important.

Suppose:

```
Northstar → same issue
LumenWorks → same issue
Customer C → same issue
Customer D → same issue
```

The system could recognize:

> **This may be a platform-wide problem rather than an individual customer problem.**

The assessment explicitly gives this as another example. CalQuity AI Engineer — Job Desc…

---

4. Why is proactive detection valuable?

Think about the difference:

Reactive system

> "Tell me what problem this customer has."

Proactive system

> "Tell me what problems the support organization should pay attention to."

The second one can potentially help the operations team **prioritize work before issues become larger**.

---

5. Problem 2 — Trust and Reliability

This is arguably the more important problem.

ParcelPilot says:

> **The team is concerned about trust.**

Why?

Because the information environment is messy.

The assessment explicitly says:

- Policies change.
- Customer contracts may override general rules.
- Different systems may disagree.
- Previous support answers may be wrong. CalQuity AI Engineer — Job Desc…

---

6. The danger: confidently wrong AI

Imagine the AI says:

> **"Yes, Northstar is entitled to a service credit."**

The answer sounds confident.

But suppose:

- It used an outdated policy.
- The customer's agreement says something different.
- The historical ticket it relied on was incorrect.

Then the AI has produced a **confidently incorrect answer**.

The assessment warns that this would quickly reduce user adoption. CalQuity AI Engineer — Job Desc…

This is why our earlier discussion about **verification** is so important.

---

7. What does "trust and reliability" actually require?

The assessment says the system should make deliberate decisions about four things:

1. Source reliability

> Which information should we trust?

2. Conflicts

> What happens when sources disagree?

3. Uncertainty

> What happens when we don't have enough evidence?

4. Human intervention

> When should the AI stop and involve a person?

CalQuity AI Engineer — Job Desc…

These are explicitly stated requirements for the broader trust problem.

---

8. This connects everything we've learned

Now the whole architecture makes more sense.

Suppose:

> "Can Northstar cancel ORD-1001 without a cancellation fee?"

The AI shouldn't just retrieve a document.

It needs to:

```
Question
   ↓
Find ORD-1001
   ↓
Identify customer
   ↓
Find Northstar agreement
   ↓
Find applicable policy/SOP
   ↓
Compare sources
   ↓
Determine which rule applies
   ↓
Check for conflict
   ↓
Determine confidence
   ↓
Answer or escalate
```

That's **Trust and Reliability in practice**.

---

9. The two problems are different

This distinction is important for your assignment.

Problem	Main question
Proactive Issue Detection	"What problems should the support team pay attention to?"
Trust & Reliability	"Can we trust the AI's investigation and decision?"

So:

Proactive detection = finding problems

Trust/reliability = making sure the AI doesn't make bad decisions

---

10. Is Problem 1 mandatory?

Here's an important distinction.

The assessment says:

> **"Beyond the minimum requirements, ParcelPilot has identified two broader problems."** CalQuity AI Engineer — Job Desc…

Then later:

> **"Required: Build the minimum requirements described above."**

And:

> **"Think Beyond the Immediate Requirements: Tell us what else you would develop..."** CalQuity AI Engineer — Job Desc…

So the two broader problems are **not the same thing as the minimum technical requirements**.

You must build the minimum requirements.

Then you need to demonstrate that you have thought about the broader product problems.

---

11. This becomes important for our project strategy

We should not make the mistake of spending all our time building a sophisticated proactive monitoring system and then failing to complete the required chatbot.

The logical priority is:

```
PRIORITY 1
Minimum working agent
       ↓
PRIORITY 2
Reliable multi-step reasoning
       ↓
PRIORITY 3
Security + confirmation + escalation
       ↓
PRIORITY 4
Additional product problem
       ↓
Proactive issue detection
```

The assessment explicitly says to **prioritize ideas and explain why they matter**.

CalQuity AI Engineer — Job Desc…

---

12. Page 7 using our framework

WHAT?

What additional problems does ParcelPilot want solved?

> Proactive issue detection + trustworthy AI decisions.

WHY?

Why?

> Reactive chat only responds after someone asks, while unreliable AI decisions can destroy trust.

WHAT IF?

What if:

- Complaints suddenly increase?
- Many tickets describe the same issue?
- High-severity tickets approach SLA?
- Multiple customers experience the same problem?
- Policies conflict?
- Customer agreements override general policies?
- Previous answers are wrong?

WHAT NEXT?

For proactive detection:

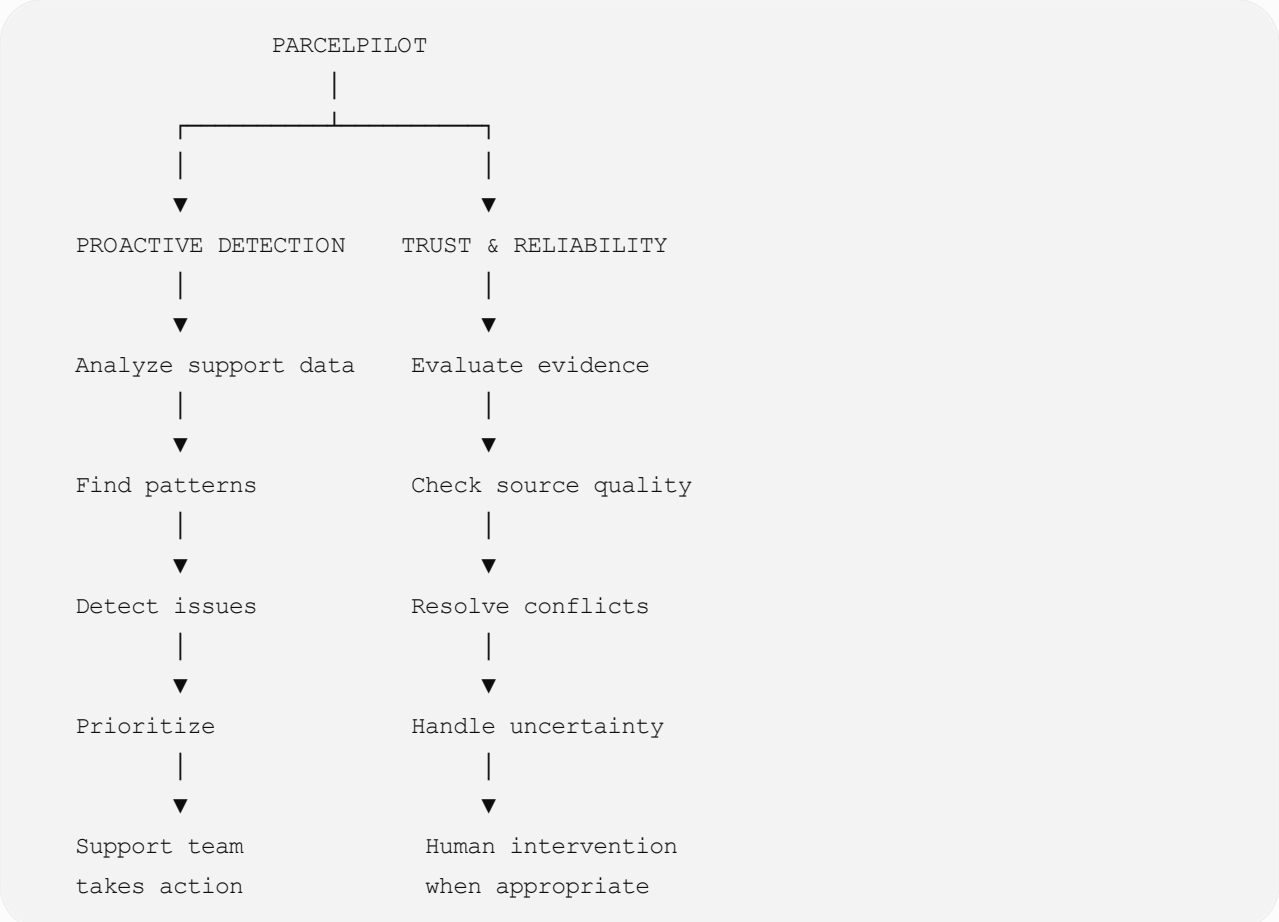
> Identify → prioritize → alert/investigate.

For trust:

> Retrieve → verify → resolve conflicts → assess uncertainty → answer or involve human.

---

13. The complete Page 7 mental model



The key lesson from Page 7

Page 7 changes our thinking from "build a chatbot" to "build a useful support system."

The chatbot is the **minimum required product**.

But ParcelPilot's bigger vision is:

> **An AI system that not only answers support questions, but also helps the operations team discover important problems proactively—and does so in a way that people can trust.**

And for our project, the safest strategy is to **finish the required agent first**, then choose one broader problem—most naturally **Proactive Issue Detection**—as the additional product feature and explain why we prioritized it. CalQuity AI Engineer — Job Desc…

Sources

Sources

page 8

Page 8 — Complete Explanation

Page 8 is essentially the **submission and evaluation roadmap**. It tells us **what you have to submit**, what documents need to accompany the application, and what the reviewers want to see.

The important point is that **Page 8 is not asking you to add random features**. It is asking you to clearly demonstrate the system you built and explain your technical/product decisions.

---

1. First: What is actually required?

The assessment says:

> **Build the minimum requirements described above.** CalQuity AI Engineer — Job Desc…

So our first priority remains:

```
WORKING AI AGENT
      ↓
Required tools
      ↓
Multi-step reasoning
      ↓
Access control
      ↓
Confirmation before actions
```

```
      ↓
Reliable handling of sources
```

Only after that should we worry about additional features.

---

2. They want you to think beyond the minimum

The assessment also says:

> **Tell us what else you would develop for ParcelPilot if you were continuing to work on the product.** CalQuity AI Engineer — Job Desc…

This is where our **Proactive Issue Detection** idea comes in.

You should explain:

> "If I had more development time, I would build X because it solves Y problem and provides Z value."

You don't necessarily need to build every additional idea.

The assessment asks you to **prioritize** your ideas and explain why they matter. CalQuity AI Engineer — Job Desc…

---

3. They are deliberately giving us design freedom

This is a very important sentence:

> They are intentionally **not specifying every aspect of the workflow, architecture, or interface**. CalQuity AI Engineer — Job Desc…

That means they don't expect everyone to build exactly the same application.

They want to see your judgment.

For example, the assessment doesn't force you to use:

- A particular LLM
- A particular vector database
- A particular frontend framework
- A particular agent framework

Instead, you need to make **sensible technical trade-offs**.

---

4. What does "technical trade-off" mean?

Suppose you have two choices.

Option A

Very sophisticated architecture.

Advantages:

- More scalable
- More sophisticated

Disadvantages:

- Takes much longer
- More components to debug
- Potentially unnecessary for a first-round assessment

Option B

Simpler architecture.

Advantages:

- Faster to build
- Easier to demonstrate
- Easier to debug

Disadvantages:

- May not scale as well

You should choose based on the problem and explain:

> **Why this architecture is appropriate for the assessment and current scope.**

That's what they mean by sensible product and technical trade-offs.

---

5. Submission Requirement 1 — Repository

You need to submit:

> **A link to the code repository.**

The repository should be public and have:

> **Clear setup and run instructions.** CalQuity AI Engineer — Job Desc…

So the reviewer should be able to understand:

```
Clone repository
      ↓
Install dependencies
      ↓
Configure environment
      ↓
Run application
      ↓
Use chatbot
```

A project that works only on your computer but cannot be reproduced is weaker.

---

6. Submission Requirement 2 — Hosted Application

A hosted version of the application is:

> **Highly preferred.** CalQuity AI Engineer — Job Desc…

That means ideally the evaluator can click a link and directly use:

```
Browser
   ↓
Your AI application
   ↓
Ask question
   ↓
See agent response
```

You don't absolutely have to interpret "highly preferred" as an absolute requirement, but **having a working hosted demo is strategically valuable**.

---

7. Submission Requirement 3 — Demo Video

You need approximately a **5-minute video**. CalQuity AI Engineer — Job Desc…

The video needs to cover three things:

1. Solution architecture

Explain:

> How does the system work?

For example:

```
User
 ↓
Chat interface
 ↓
Agent
 ↓
Document search / Data tool / Action tool
 ↓
Verification
 ↓
Answer / Action
```

---

2. Working application

Actually demonstrate it.

Don't just show code.

For example:

> "Can Northstar cancel ORD-1001 without a cancellation fee?"

Then show the agent:

```
Identify order
      ↓
Retrieve agreement
      ↓
Check policy
      ↓
Reason
      ↓
Answer
```

---

3. Important decisions and why

This is where you explain:

> **Why did I build it this way?**

For example:

> "I separated document retrieval from structured-data access because policies and operational records have different retrieval requirements."

Or:

> "I require confirmation before state-changing actions to prevent unintended ticket updates."

The assessment explicitly asks for the important product/technical decisions and the reasons behind them. CalQuity AI Engineer — Job Desc…

---

8. Submission Requirement 4 — Architecture Note

This is a short technical document.

It needs to cover:

Agent design

How does the agent reason and orchestrate tools?

Tool design

What tools exist and what does each one do?

Document + structured-data handling

How do you handle:

- PDFs/documents
- Excel/account/order/ticket data?

Source reliability + conflict handling

How do you handle:

- Current vs deprecated information?
- Customer-specific rules?
- Conflicting sources?
- Historical information?

Major technical trade-offs

Why did you choose your architecture and approach?

These requirements are explicitly listed in the assessment. CalQuity AI Engineer — Job Desc…

---

9. Submission Requirement 5 — Product Note

This is different from the Architecture Note.

Architecture Note =

> **How did you build it?**

Product Note =

> **Why did you build it this way and what would you do next?**

The Product Note must cover:

1. Which additional client problem you chose.
2. How you addressed it.
3. What else you would build.
4. What you intentionally left out.
5. One metric you would use to judge whether the product is useful. CalQuity AI Engineer — Job Desc…

---

10. "What you intentionally left out" is important

This is actually a good product-engineering question.

You don't need to pretend:

> "My system solves everything."

Instead, say:

> "I intentionally did not build X because it was outside the minimum scope / required more data / would add complexity without enough value at this stage."

That demonstrates **scope management**.

For example:

```
Required chatbot
       ↓
BUILD

Proactive issue detection
       ↓
BUILD / prototype

Full production monitoring platform
```

```
       ↓
NOT BUILD NOW
```

The important thing is to explain **why**.

---

11. One useful metric

They specifically ask for:

> **One metric you would use to judge whether the product is useful.**
> CalQuity AI Engineer — Job Desc…

This means we shouldn't simply say:

> "The AI is accurate."

We need to think about business/product usefulness.

For example, a possible metric could be:

> **Percentage of eligible support requests resolved without human escalation, while maintaining an acceptable accuracy/reliability threshold.**

But this is **our proposed metric**, not a metric mandated by the assessment.

We should choose the final metric based on the feature we actually build.

---

12. Submission Requirement 6 — AI Tool Usage

You also need to briefly state:

> **Which AI coding tools you used and how you used them.** CalQuity AI Engineer — Job Desc…

So we shouldn't hide AI assistance if we use it.

We can document it clearly, for example:

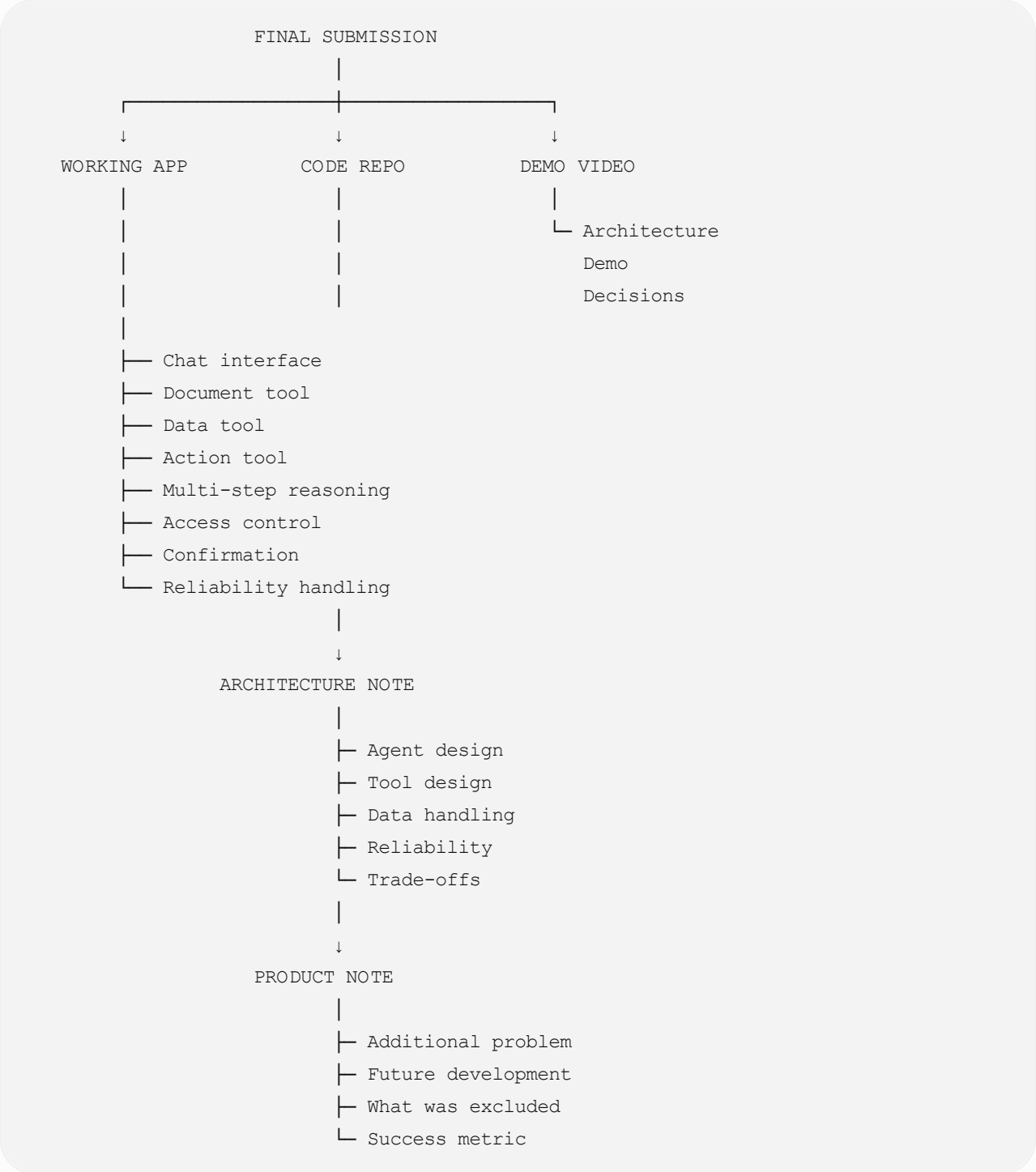
```
AI coding assistant
       ↓
Used for:
- Code scaffolding
- Debugging
- Documentation
- Refactoring
```

The exact tools we actually use should be reported honestly.

---

13. Page 8 gives us the final project structure

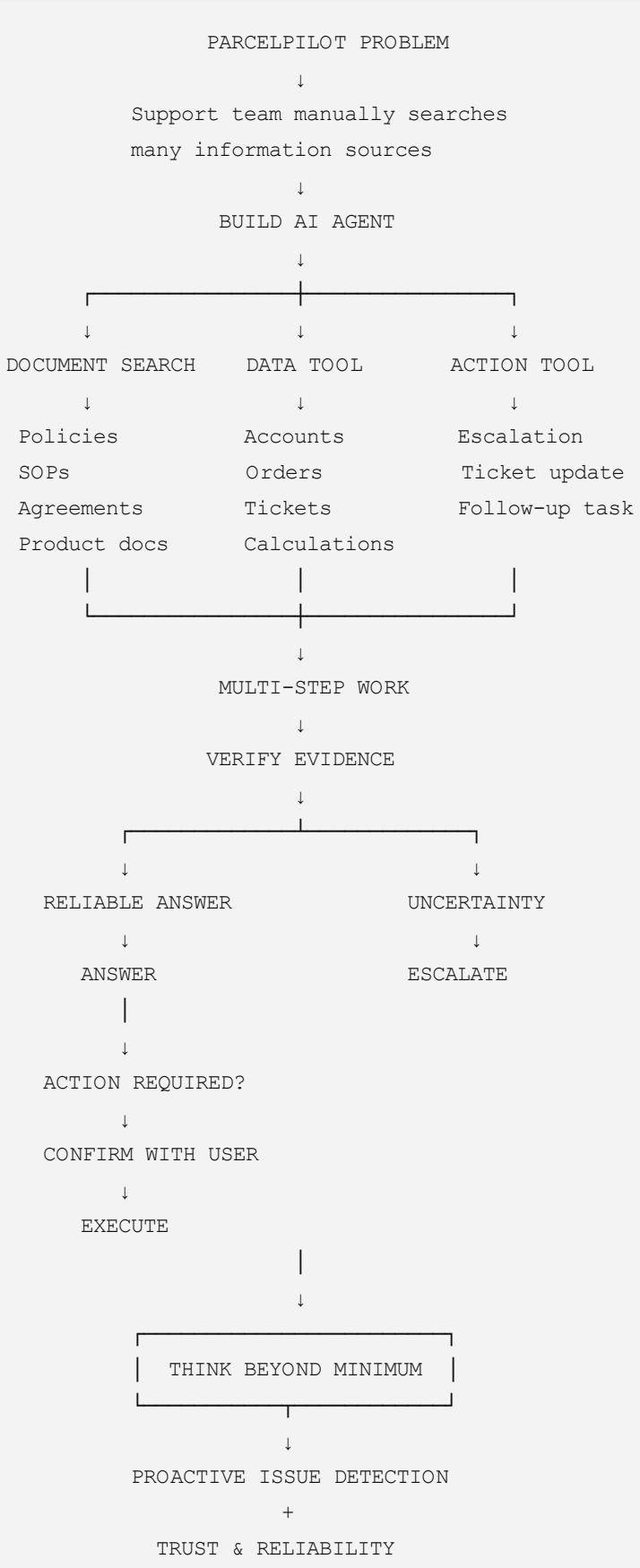
Now we can see the complete deliverable:



---

14. The entire 8-page assessment in one map

Now that we've gone through all 8 pages, the whole assignment can be reduced to this:



The most important thing from Page 8

Page 8 is telling us how to prove that the system is good.

Not just:

> "Here is my code."

But:

> **Here is the working application → here is how it is architected → here is why I made these decisions → here is the additional problem I chose → here is what I deliberately didn't build → and here is how I would measure whether the product is useful.**

The assessment therefore tests **three things at once**:

1. Engineering

Can you actually build the agent?

2. AI/system thinking

Can you make retrieval, tools, reasoning, reliability, and actions work together?

3. Product judgment

Can you decide what matters, what to build, what not to build, and how to measure value?

That is the complete meaning of **Page 8**. CalQuity AI Engineer — Job Desc…

Sources

Sources